Software Requirements Specification

Gibiyachin (ግቢያችን)

University Marketplace and Campus Services Platform

Document identifier	GIB-SRS-2026-V2.1
Version	2.1
Date	October 2026
Prepared by	Eyob Arega

Document Information

Document identifier	GIB-SRS-2026-V2.1
Version	2.1
Publication date	October 2026
Prepared by	Eyob Arega
Target environment	Ethiopian higher education institutions
Repository	Gibiyachin full-stack repository (backend/ and frontend/); the legacy internal package name is “campushub”
Reference frameworks	IEEE Std 830-1998 and ISO/IEC/IEEE 29148:2018, used as guidance only

Revision History

Version	Date	Author	Description of changes
1.0	Sep 2026	Eyob Arega	Preliminary requirements for the Phase 1 graduation MVP.
1.1	Sep 2026	Eyob Arega	Added Phase 2 features: Redis, manual payment abstraction, SMS logs.
1.2	Oct 2026	Eyob Arega	Added Phase 3 multi-university tenancy and university isolation.
2.0	Oct 2026	Eyob Arega	Revised to match the implemented system: 40 database tables, manual payment proof verification, 5.0% / 2.5% / 0% commission rules, HMAC-SHA256 ticketing with check-in and reversal, deep-link notifications, Sharp image sanitization, role-adaptive profiles, and the business model.
2.1	Oct 2026	Eyob Arega	Re-checked against the repository (323 backend tests, lint, build, benchmark). Corrected endpoints, CSRF/JWT/cookie statements, and migrations. Added FR-043 to FR-052, Section 2.7, Section 14.3, and Section 15.2.

Summary

Gibiyachin (ግቢያችን) is a multi-tenant web application designed for Ethiopian university campuses.

Campus commerce, equipment sharing, and student events often rely on informal channels such as physical bulletin boards, Telegram groups, and cash handovers. These channels offer little verification or oversight, and users can be exposed to problems such as forged event tickets, equipment that is never returned, and fraudulent sales.

Gibiyachin brings these activities into one verified, structured platform. It provides five main services:

1. **Merchant-to-Student Marketplace:** Stores run by campus-adjacent merchants or students, with product catalogs, stock holds, confirmation of physical order handover by both parties, and platform commission settlement.
2. **Student-to-Student (P2P) Exchange:** Peer-to-peer listings for textbooks, calculators, electronics, and dorm essentials, with content reporting and moderation.
3. **Campus Resource and Equipment Rentals:** Equipment borrowing between students, with availability date ranges, refundable deposits, pickup and return confirmations, and tiered late fees.
4. **Digital Event Ticketing and Admission Control:** Event management with Campus Administrator review, ticket tiers with quotas, manual payment proof approval, single-use signed QR codes (HMAC-SHA256), human-readable ticket numbers (GIB-XXXXXXXX), camera-based scanning at the gate, and a check-in audit log that supports reversals.
5. **Multi-Tenant Campus Administration:** Data is separated by university, so that students, merchants, and organizers see only the data of their own campus. Each campus is managed by Campus Administrators, and Super Administrators manage the platform as a whole.

This document specifies the Gibiyachin system, which uses a PostgreSQL database, an Express.js backend, and a React/Vite frontend. Section 14 and Section 15 state which features have automated test evidence and which do not.

Table of Contents

Summary ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Project Scope 2
1.5 Definitions, Acronyms, and Abbreviations 2
1.6 References 3
2. Overall Description 4
2.1 Product Perspective 4
2.2 Product Functions Overview 4
2.3 User Classes and Characteristics 5
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 5
2.6 Assumptions and Dependencies 6
2.7 External Interface Requirements 6
3. User Roles and RBAC Permission Matrix 7
3.1 Role Definitions 7
3.2 Role-Based Access Control (RBAC) Matrix 7
3.3 Mapping to IEEE 830 and ISO/IEC/IEEE 29148 Topics 8
4. System Architecture and Technical Specifications 10
4.1 Architectural Pattern 10
4.2 Technology Stack Details 11
4.3 Multi-Tenant Isolation and Scoping Model 11
4.4 Security Infrastructure and Session Lifecycle 12
5. Functional Requirements 13
5.1 Authentication, Identity and Access Control 13
5.2 User Profiles and Account Management 16
5.3 Merchant Marketplace and Product Catalog 18
5.4 Order Processing and Physical Handover Lifecycle 20
5.5 Peer-to-Peer (P2P) Student Marketplace 21
5.6 Borrowing and Equipment Rental System 22
5.7 Campus Event Management 23
5.8 Digital Ticketing, Cryptographic QR and Entry Validation 24
5.9 Payment Processing and Commission Management 26
5.10 Actionable Notifications and Campus Verification 27
5.11 Platform Governance and Multi-Tenancy 27
5.12 Extended Platform Capabilities 28
6. Non-Functional Requirements (NFRs) 32
6.1 Performance Requirements 32
6.2 Security and Data Protection Requirements 32
6.3 Reliability and Availability Requirements 33
6.4 Usability and Accessibility Requirements 33
6.5 Maintainability and Scalability Requirements 33

7. Database Design and Entity Relationship Specifications 34
7.1 Database Entity Catalog (40 Tables) 35
8. System Process Flows and Data Flow Diagrams 38
8.1 Level 0: Context Data Flow Diagram 38
8.2 Level 1: Subsystem Data Flow Diagram 38
8.3 Level 2: Core Transaction Process Flows 40
9. Core Use Cases 42
UC-01: User Registration and Email OTP Verification 42
UC-02: User Login and Token Refresh Rotation 42
UC-03: Profile Customization and Privacy Shielding 42
UC-04: Merchant Product Creation and Stock Tracking 43
UC-05: Order Placement and Physical Handover Handshake 43
UC-06: Order Cancellation and Stock Restocking 43
UC-07: P2P Classified Listing and Negotiation 44
UC-08: Equipment Rental Booking and Return Lifecycle 44
UC-09: Event Proposal and Campus Admin Review 44
UC-10: Ticket Purchase and Manual Payment Verification 44
UC-11: Gate Admission Scan and Repeat Entry Denial 45
UC-12: Admission Reversal 45
UC-13: Merchant Commission Settlement 45
UC-14: Campus Verification Document Review 45
UC-15: Super Admin University Provisioning 46
10. Security Architecture and Cryptographic Controls 47
10.1 Authentication and Session Security 47
10.2 Cryptographic QR Code Verification and Key Rotation 47
10.3 Insecure Direct Object Reference (IDOR) and Tenant Isolation 47
10.4 Payment Verification and Rate Limiting 48
11. File Upload and Storage Management 49
11.1 Processing Pipeline 49
12. Business Model Canvas and Revenue Streams 50
12.1 Revenue Streams Breakdown 50
13. Requirements Traceability Matrix (RTM) 51
14. Implementation Status and Future Roadmap 54
14.1 Implemented Features and Evidence 54
14.2 Future Enhancements (Deferred to Subsequent Phases) 55
14.3 Known Limitations and Hardening Backlog 56
15. Acceptance Criteria and System Verification Summary 57
15.1 Test Execution Summary 57
15.2 Non-Functional Requirement Verification Status 58
15.3 Acceptance Criteria for the Pilot 59
16. Conclusion 59

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the **Gibiyachin (ግቢያ ችን) Platform**. It is written for the course instructor, the developers, and the testers. It explains the main workflows, the database structure, the security controls, and the business rules of the system, and it shows how each requirement relates to use cases, database tables, and test evidence. IEEE Std 830-1998 [1] and ISO/IEC/IEEE 29148:2018 [2] were used as guidance for organizing this document; the document makes no claim of formal compliance with either standard. Statements about the implementation are based on the project repository. Section 14 and Section 15 state which features have automated test evidence and which do not.

1.2 Document Conventions

- **Requirement Identifiers:** Functional requirements are designated with the prefix FR-XXX (e.g., FR-001, FR-041). Non-functional requirements are designated with NFR-XXX.
- **Priority Ratings:** Each functional requirement has a priority rating:
  - **High:** Essential core capability without which the platform cannot operate.
  - **Medium:** Significant capability needed for operational efficiency and trust.
  - **Low:** Value-added capability or non-blocking extension.
- **Requirement Wording:** The non-functional requirements use the RFC 2119 keywords MUST, SHALL, SHOULD, and MAY. Functional requirements are written as descriptions of system behavior.
- **Implementation and Test Status:** Listing a requirement in Section 5 does not by itself mean that it has been tested. Sections 13, 14, and 15 show what implementation and test evidence exists for each requirement.
- **Terminology:** “Campus Administrator” is the role named `admin` in the database, and “event staff” means organizers, Campus Administrators, and Super Administrators who scan tickets. “P2P listing” means a second-hand item sold between students; equipment rentals are described separately in Section 5.6.

1.3 Intended Audience and Reading Suggestions

1. **Instructor and Evaluators:** Read Sections 1, 2, 4, 12, 14, and 15 to assess the architecture, the business model, and the verification status.
2. **Software Engineers and Developers:** Sections 4, 5, 7, 8, 10, and 13 give API details, state changes, security controls, and database structure.
3. **Testers:** Sections 5, 6, 13, and 15 link requirements to the tests and checks that cover them.
4. **Campus Administrators and Stakeholders:** Sections 3, 5.9, 5.10, 5.11, and 12 cover governance, payment workflows, and operating rules.

1.4 Project Scope

Gibiyachin is a full-stack, responsive web application that provides campus services in one place. The system covers the following 13 areas:

- Dual-path identity verification (academic domain matching and physical ID document upload).
- Role-adaptive user profiles with privacy controls and activity feeds.
- Merchant stores with multi-image product listings and stock locking.
- Two-party confirmed physical orders with 5.0% merchant commission.
- Peer-to-peer student marketplace with condition grading and contact request workflows.
- Equipment rental management with deposit holds, pickup/return confirmations, and late fees.
- Campus event planning with university administrative pre-approval.
- Multi-tier digital event ticketing with manual payment proof approval, human ticket numbers, and HMAC-SHA256 signed QR admissions.
- Actionable in-app notifications with allowlisted deep-link routing.
- Multi-tenant data isolation partitioning campus data by university_id.
- Commission tracking, manual payment settlement, and auditable financial records.
- Comprehensive administrative dashboards for Campus Admins and Super Admins.
- File uploads that are checked by file signature (magic bytes) and re-encoded as images.

Section 5.12 describes further implemented capabilities: ticket waitlists, reviews and ratings, content reporting and moderation, featured-product promotion, offline check-in, CSV export, recurring events, in-app payment for P2P orders, fee schedules and agreement versions, and account deactivation. The following are outside the current scope: native mobile applications, in-app chat, an escrow wallet, live SMS delivery, and a validated online payment gateway (Section 14.2).

1.5 Definitions, Acronyms, and Abbreviations

Table 1: Definitions, acronyms, and abbreviations

Term / Acronym	Definition
Gibiyachin (ግቢያችን)	Amharic for “Our Campus”; the name of the platform.
AASTU	Addis Ababa Science and Technology University (primary pilot university).
RBAC	Role-Based Access Control. Server-enforced permissions governing API routes.
JWT	JSON Web Token. Stateless bearer token carrying user claims (user ID, role, university ID).
CSRF	Cross-Site Request Forgery. Protected via double-submit / header token comparison.
HMAC-SHA256	Hash-Based Message Authentication Code using SHA-256 for cryptographic QR signing.
OTP	One-Time Password. 6-digit numeric token for email ownership verification.
P2P	Peer-to-Peer. Direct student-to-student transactions.
IDOR	Insecure Direct Object Reference. Access-control weakness that the system addresses with ownership and tenant checks.
Sharp / Libvips	Image processing library used to decode, sanitize, and re-encode images.

Term / Acronym	Definition
CBE	Commercial Bank of Ethiopia (primary manual payment receiving account).
Telebirr	Mobile money payment service operated by Ethio Telecom.
Campus Administrator	University-level administrator (database role admin); also written “Campus Admin” in tables.
Event staff	Users who may scan tickets at an event: organizers, Campus Administrators, and Super Administrators.
Tenant	One university and its data. Campus data carries a university_id.
Booking	A rental booking; also the transaction type “booking” in payments and commissions.
ETB	Ethiopian Birr, the currency used for all amounts.

1.6 References

1. IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
2. ISO/IEC/IEEE 29148:2018: *Systems and software engineering — Life cycle processes — Requirements engineering*.
3. RFC 7519: *JSON Web Token (JWT)*.
4. RFC 2104: *HMAC: Keyed-Hashing for Message Authentication*.
5. OWASP Top 10 Application Security Risks (2021 Edition).
6. Gibiyachin project repository (Express.js backend, React frontend, PostgreSQL schema and migrations).

2. Overall Description

2.1 Product Perspective

Gibiyachin is a single, self-contained multi-tenant web application. Unlike open public classifieds (e.g., Jiji, Telegram groups) or global e-commerce systems (e.g., Amazon, Shopify), Gibiyachin is intentionally scoped around the physical and social boundaries of Ethiopian university campuses.

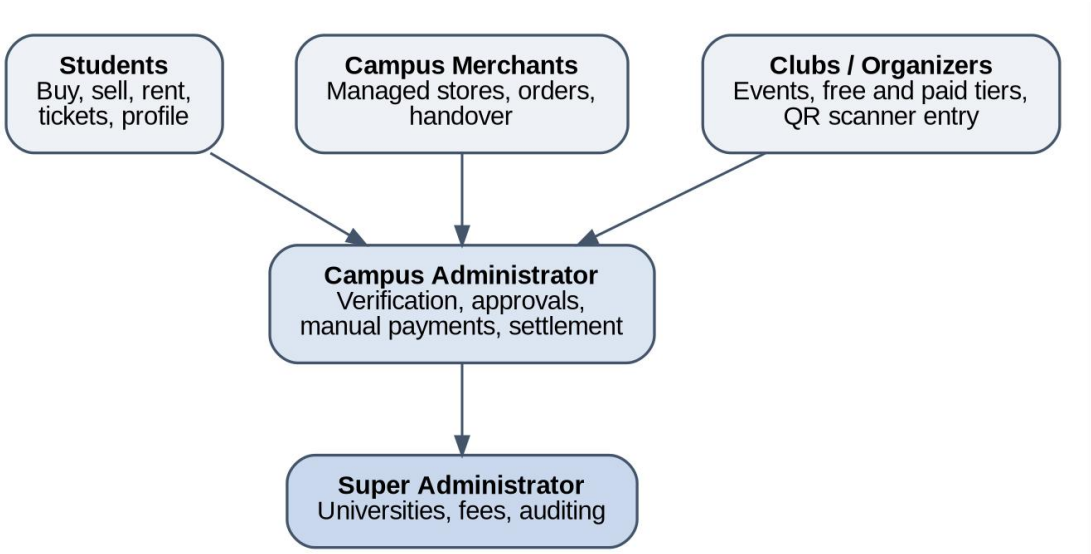


Figure 1: Gibiyachin user groups and administrative hierarchy

2.2 Product Functions Overview

- **Authentication & Security:** Registration, domain matching, 6-digit OTP verification, login, HttpOnly cookie refresh rotation, CSRF header validation, password reset with 1-hour hashed tokens.
- **Identity & Profiles:** Role-adaptive profiles with cover/avatar customization, academic year/department fields, privacy masking for cross-campus users, unified activity feeds, and seller/organizer public showcases.
- **Marketplace & Inventory:** Storefronts, categories, atomic stock decrements, image arrays, and merchant sales dashboards.
- **Physical Handover Order Lifecycle:** Status transitions (pending_payment -> confirmed -> ready -> completed), merchant payment receipt confirmation, and buyer physical receipt confirmation.
- **Peer-to-Peer (P2P) Exchange:** Used item listings with condition grading, contact requests, and buyer-seller negotiations.
- **Resource Rentals:** Daily rental bookings, deposit calculations, pickup/return confirmations, tiered late-fee policies, and dispute tracking.
- **Event Management & Ticketing:** Admin event approval, multiple ticket types, quotas, ticket numbers, HMAC-SHA256 QR code generation, camera QR scanning, manual ticket lookup, and attendee check-in with reversals.

- **Financial & Commission Management:** 5.0% merchant order commission, 2.5% event ticket commission (97.5% organizer payout), 0% rental commission, official platform receiving accounts, manual payment proof submission, and admin settlement.
- **Actionable Notifications:** In-app notifications with allowlisted deep-link navigation.
- **Campus Verification:** Fast-track email domain verification plus manual student ID review queue.
- **Extended Capabilities:** Ticket waitlists, reviews and ratings, content reporting and moderation, featured-product promotion, offline check-in, CSV export, recurring events, in-app P2P payment, fee schedules and agreement versions, and account deactivation (Section 5.12).

2.3 User Classes and Characteristics

1. **Students (General Campus Users):** Undergraduate and graduate students. Primary buyers, renters, ticket attendees, and P2P sellers. Many are expected to use mobile phones, so the interface needs to work on small screens and slow connections.
2. **Campus Merchants:** Commercial vendors (bookstores, print shops, cafes, electronics suppliers). Manage stores, fulfill orders, confirm payment receipts, and settle platform commissions.
3. **Student Club Leads & Event Organizers:** Department associations, academic societies, and cultural clubs. Create events, manage ticket tiers, track revenue, and scan QR tickets at doors.
4. **Campus Administrators (University Admins):** Designated university personnel or student union officials. Review campus verifications, approve events, review merchant applications, verify ticket payments, resolve disputes, and settle merchant commissions.
5. **Super Administrators:** Platform operators. Onboard universities, configure institution parameters, assign campus admins, manage global fee schedules, and audit platform security logs.

2.4 Operating Environment

- **Server:** A Linux server running Node.js 20 or higher. The automated tests were run on Ubuntu 24.04 with Node.js v22.22.2 (Section 15.1). No test evidence is recorded for other operating systems.
- **Database:** PostgreSQL 15 or higher with the pgcrypto extension. The automated tests were run on PostgreSQL 16.13.
- **Client Devices:** Target browsers are Google Chrome 110+, Mozilla Firefox 110+, Apple Safari 16+, and Microsoft Edge, on mobile (Android and iOS) and desktop screens. Cross-browser testing has not been recorded.
- **Hosting:** A suggested minimum of 2 GB RAM, 2 vCPUs, and 20 GB SSD storage, with a public directory (`/uploads`) and a private directory (`/private_uploads`) for files. These figures have not been validated by load testing.

2.5 Design and Implementation Constraints

1. **Payment Practices in Ethiopia:** The project assumes that few students use online card payments (for example Visa or Mastercard). The pilot therefore relies on direct bank or mobile-money transfers (CBE, Telebirr, Awash Bank), confirmed through transaction references and payment proof screenshots.

2. **Single-Monolith Deployment:** Backend is structured as a modular monolith in Express.js with modular routing, centralized error handlers, and transaction-wrapped service operations.
3. **Network Latency & Bandwidth:** Campus Wi-Fi and mobile data networks exhibit fluctuating bandwidth. The frontend is built with Vite, and uploaded images are re-encoded on the server (Section 11). The size of the production bundle is reported in Section 15.1.
4. **Tenant Scoping:** All database entities (except Super Admin global catalogs) must strictly bind to a university_id foreign key.

2.6 Assumptions and Dependencies

- **Email Service Availability:** Standard SMTP mailer (Nodemailer) is available for OTPs and password resets. In environments without an active SMTP server, verification codes and reset links are recorded in development audit logs.
- **Camera Availability:** Mobile and laptop devices used by event staff have a functional camera supported by html5-qrcode.
- **Institutional Identity:** University administrators are assumed to review student ID proofs and resolve campus disputes honestly and correctly.

2.7 External Interface Requirements

This section specifies the interfaces between Gibiyachin and its users, devices, external software and networks.

- **User Interfaces:** A responsive single-page React application usable from 320 px to 2560 px viewports (NFR-015), with role-gated routes, colour-coded status indicators (NFR-016), and a camera-based QR scanner screen for event staff.
- **Hardware Interfaces:** No dedicated hardware is required. Staff devices that scan tickets need a working camera; the server hardware minimum is given in Section 2.4.
- **Software Interfaces:** PostgreSQL 15 or higher through the pg driver using parameterized queries; an SMTP server through Nodemailer for verification codes and reset links; Redis through ioredis as an optional store for rate limiting and caching, with automatic fallback to in-process behaviour; Sharp (libvips) for image processing.
- **Payment Interface:** Payments pass through a provider abstraction (services/payments.js) selected by PAYMENT_PROVIDER. The pilot operates on manual, proof-based payments (CBE, Telebirr, Awash Bank); a mock provider supports development; a chapa provider and a webhook endpoint (/api/payments/webhook) are implemented but not validated against the live gateway and are disabled by default.
- **Messaging Interface:** In-app notifications are real. SMS delivery is simulated through notify.sendSms until an Ethio Telecom gateway is contracted.
- **Communications Interfaces:** An HTTPS JSON REST API under /api; refresh and CSRF cookies scoped as described in Section 4.4; tenant selection for public visitors through the X-University-Id or X-University-Slug headers or the universityId query parameter; security headers through Helmet; public media served from /uploads and private proofs served only through authenticated endpoints.

3. User Roles and RBAC Permission Matrix

3.1 Role Definitions

Gibiyachin defines five roles in a PostgreSQL enum named `user_role`. Roles are checked on the server. In this document, the role `admin` is called Campus Administrator (or Campus Admin) and `super_admin` is called Super Administrator.

- ‘student’
- ‘merchant’
- ‘organizer’
- ‘admin’
- ‘super_admin’

Every request to a protected endpoint passes through `requireAuth()` and role-checking middleware (`requireRole(...)`).

3.2 Role-Based Access Control (RBAC) Matrix

Table 2: Role-based access control matrix

Functional Capability	Student	Merchant	Organizer	Campus Admin	Super Admin
Register & Verify Account	Yes	Yes	Yes	Invite/Create	Direct
Manage Own Profile & Privacy	Yes	Yes	Yes	Yes	Yes
Browse Products, Rentals & Events	Yes	Yes	Yes	Yes	Yes
Place Merchant Order	Yes	No	No	Optional	No
Confirm Physical Material Receipt	Yes	No	No	Override	Override
Create & Manage Store / Products	No	Yes	No	No	View/Mod
Confirm Order Payment Receipt	No	Yes	No	Yes	Yes
Advance Order (Ready for Pickup)	No	Yes	No	Yes	Yes
Create P2P Listing / Rental Item	Yes	Yes	Yes	Mod	Mod
Submit Merchant / Organizer App	Yes	Yes	Yes	Review	Review
Create Events & Ticket Tiers	No	No	Yes	Yes	Yes
Approve / Reject Events	No	No	No	Yes	Yes
Purchase Event Ticket & Pay	Yes	Yes	Yes	Yes	No
Verify Ticket Manual Payment	No	No	No	**Yes**	**Yes**

Functional Capability	Student	Merchant	Organizer	Campus Admin	Super Admin
Scan & Validate Ticket (QR / ID)	No	No	**Yes**	**Yes**	**Yes**
Reverse Admitted Ticket	No	No	**Yes**	**Yes**	**Yes**
View Own Commissions Ledger	No	Yes	Yes	All Sellers	All Campuses
Submit Commission Payment Proof	No	Yes	No	Review	Review
Settle Commission Payments	No	No	No	**Yes**	**Yes**
Review Campus ID Documents	No	No	No	**Yes**	**Yes**
Resolve Disputes & Moderation	Report	Report	Report	**Yes**	**Yes**
Create / Manage Universities	No	No	No	No	**Yes**
Assign Campus Administrators	No	No	No	No	**Yes**
Manage Fee Schedules & Agreement Terms	No	No	No	No	**Yes**
Join Ticket Waitlist (Sold-Out Tier)	Yes	No	No	No	No
Submit Review (Completed Transaction)	If eligible	If eligible	If eligible	If eligible	If eligible
Report Content	Yes	Yes	Yes	Yes	**Yes**
Resolve Reports & Moderate Content	No	No	No	Yes	**Yes**
Promote Product (Featured Placement)	No	Yes	No	No	No
Batch Check-In (Offline Scan Sync)	No	No	Yes	Yes	**Yes**
Generate Recurring Event Occurrences	No	No	Yes	No	No
Pay P2P Order In-App	Buyer only	Buyer only	Buyer only	Buyer only	Buyer only
Export CSV Data	No	Own store orders	No	Yes	**Yes**
Deactivate / Delete Accounts	No	No	No	Yes	**Yes**

3.3 Mapping to IEEE 830 and ISO/IEC/IEEE 29148 Topics

This SRS is organized around the platform's domains rather than following the IEEE 830 template clause by clause. Table 3 shows where this document covers each topic of IEEE Std 830-1998 (Section 3, Specific Requirements) and ISO/IEC/IEEE 29148. The standards are used as guidance only.

Table 3: Mapping of this SRS to IEEE 830 and ISO/IEC/IEEE 29148 topics

IEEE 830 / 29148 clause	Location in this SRS
External interface requirements	2.7 External Interface Requirements
Functional requirements	5 Functional Requirements (FR-001 to FR-052); 8 Process Flows; 9 Core Use Cases
Performance requirements	6.1 Performance Requirements (NFR-001 to NFR-004)
Logical database requirements	7 Database Design and Entity Relationships
Design constraints	2.5 Design and Implementation Constraints; 4 System Architecture
Software system attributes: reliability, availability, security, maintainability, portability	6.2 to 6.5 Non-Functional Requirements; 10 Security Architecture
Organizing specific requirements by user class and feature	3 User Roles and RBAC Matrix; 5 Functional Requirements (grouped by feature)
Product perspective, functions, user characteristics, constraints, assumptions	2 Overall Description (2.1 to 2.6)
Definitions, references, document conventions	1.2 Document Conventions; 1.5 Definitions; 1.6 References
Traceability	13 Requirements Traceability Matrix
Verification and acceptance	15 Acceptance Criteria and System Verification Summary
Known limitations	14.3 Known Limitations and Hardening Backlog

4. System Architecture and Technical Specifications

4.1 Architectural Pattern

Gibiyachin is built as a modular monolith: one deployable backend divided into domain modules, a separate React client, and a PostgreSQL database. Figure 2 shows the three tiers.

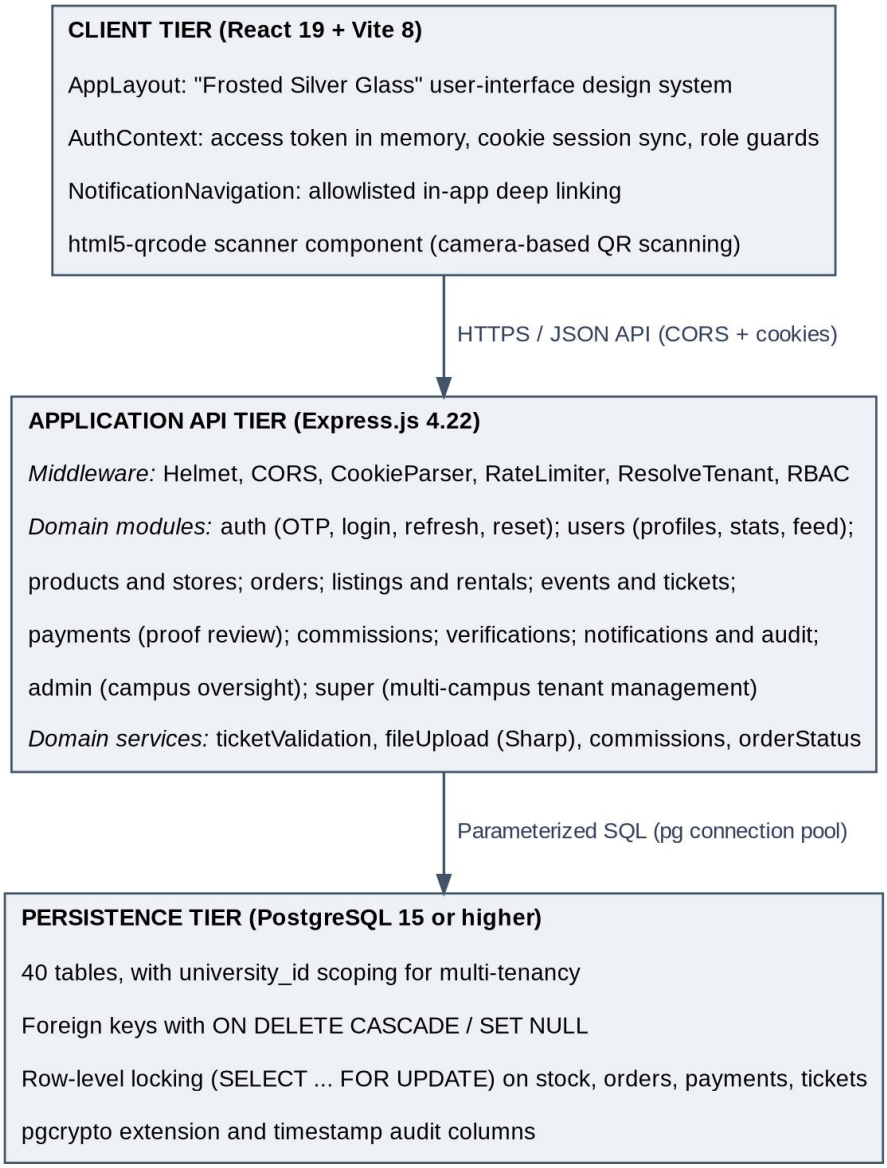


Figure 2: Three-tier architecture of Gibiyachin

4.2 Technology Stack Details

Table 4: Technology stack

Layer	Component	Version	Purpose
Frontend	React	^19.2.8	Component-based user interface library.
Frontend	Vite	^8.2.2	Development server and production bundler.
Frontend	Tailwind CSS	^4.3.3	Utility-first CSS used for the “Frosted Silver Glass” design and the responsive layout.
Frontend	React Router	^7.18.3	Client-side routing, role-gated routes, URL parameters.
Frontend	Axios	^1.20.0	HTTP client that sends the CSRF token and cookies.
Frontend	html5-qrcode	^2.3.8	Cross-platform camera scanner for QR admissions.
Backend	Node.js	20 or higher	JavaScript runtime for the backend.
Backend	Express.js	^4.22.2	Web framework: routing, middleware, and the JSON API.
Backend	PostgreSQL (pg)	^8.23.0	PostgreSQL client with connection pool and transactions.
Backend	Zod	^4.5.4	Runtime validation of request body, query, and parameters.
Backend	Bcrypt.js	^3.0.3	One-way salt hashing (cost factor 10) for stored passwords.
Backend	JsonWebToken	^9.0.3	Stateless signed access token authorization.
Backend	Sharp	^0.35.5	Libvips-backed raster image decoding, EXIF sanitization, re-encoding.
Backend	Multer	^2.3.0	Multipart/form-data streaming to secure temporary staging storage.
Backend	QRCode	^1.5.4	Server-side QR code generation (PNG and SVG data URLs).
Backend	Nodemailer	^6.10.1	SMTP delivery for verification OTPs and password reset links.
Backend	Helmet	^8.3.0	HTTP security headers (nosniff, X-Frame-Options, HSTS).

4.3 Multi-Tenant Isolation and Scoping Model

Gibiyachin uses a shared-database, shared-schema multi-tenancy model. All universities share the same tables, and each row carries a `university_id`:

1. **Tenant Anchor (universities table):** Every institution has a record with id, name, slug, domain, permitted_domains, status, feature_flags, and isolation_mode.
2. **Tenant Resolution Middleware (resolveTenant):**
   – For authenticated users (Student, Merchant, Organizer, Campus Admin), req.tenant.universityId is strictly extracted from the database record of req.user.university_id. Any client-supplied X-University-Id header that conflicts with their institutional affiliation is rejected with a 403 TENANT_MISMATCH.

- Super Administrators default to cross-tenant scope (universityId = null), but may pass an explicit X-University-Id header to drill into a single campus.
- For unauthenticated public visitors, tenant is resolved via X-University-Id, X-University-Slug, or ?universityId= query parameter. If only one university exists in the system, it defaults to that pilot university.

3. **Database Scoping Enforcement:**
   - Every domain query appends university_id = $X to filter data.
   - Domain mutations automatically populate university_id from req.tenant.universityId.
   - The test suite contains 84 multi-tenant isolation tests (tenant_integrity.test.js, tenant_auth.test.js, tenant_route_scoping.test.js, and migration_013.test.js) that check tenant isolation. All 84 passed in the recorded run (Section 15.1); passing tests do not prove that leakage is impossible.

4.4 Security Infrastructure and Session Lifecycle

- **Access Tokens:** JSON Web Token [3] signed with JWT_ACCESS_SECRET, 15-minute expiration, containing sub (user ID), role, and universityId.
- **Refresh Tokens:** Cryptographically random 64-character tokens stored as SHA-256 hashes in the refresh_tokens table with 30-day expiration. Sent exclusively in an HttpOnly, SameSite=Lax, Path=/api/auth cookie named ch_refresh.
- **CSRF Defense:** Cookie-authenticated requests (refresh and logout) require an anti-CSRF token (ch_csrf cookie matched with X-CSRF-Token header) compared via crypto.timingSafeEqual. Data-mutating API calls authenticate with a Bearer access token held in memory, which browsers never attach automatically, so they are not exposed to CSRF.
- **Refresh Rotation & Theft Detection:** Every /api/auth/refresh revokes the old refresh token and issues a new token pair. If an already-revoked refresh token is presented, the system flags token reuse, revokes all active tokens in that family_id, and forces re-authentication across all user devices.

5. Functional Requirements

This section lists 52 functional requirements (FR-001 to FR-052). Each has an identifier, priority, actor, and description, and, where relevant, preconditions, flows, and postconditions. The requirements describe what the system does; API paths and data fields are included where they help the reader find the behavior in the code. Not every requirement has automated test coverage: Table 6 in Section 13 shows the evidence for each one, and Section 14.1 summarizes the status.

5.1 Authentication, Identity and Access Control

FR-001: User Registration with University Affiliation

Priority: High

Actor: Unauthenticated Visitor (prospective Student).

Preconditions: The visitor does not possess an active account with the provided email.

Description: Allows a user to create an account by providing full name, email, password, optional phone number, desired role (student by default; the merchant role is granted only after an approved merchant application, see FR-012), and optional universityId.

Main Flow:

1. User submits registration details via POST /api/auth/register.
2. System extracts email domain (e.g., aastu.edu.et) and checks universities table (both primary domain and permitted_domains array).
3. If domain matches an active university:
   - Sets user's university_id to that university.
   - Sets verified = true and verification_method = 'domain'.
4. If email domain is a generic provider (e.g., gmail.com):
   - Validates that user provided an explicit universityId.
   - Sets user's university_id to the chosen university.
   - Sets verified = false and verification_method = 'manual_pending'.
5. Password is salted and hashed using Bcrypt (cost factor 10).
6. Creates user record in users with email_verified = false.
7. Issues a 6-digit numeric OTP via issueSignupOtp and stores its SHA-256 hash in email_otps.
8. Dispatches verification code via email (mailer.sendMail).
9. Returns 201 Created with emailVerificationRequired: true.

Alternative / Exception Flows:

- *Email Already Registered:* System aborts with 409 EMAIL_TAKEN.
- *University Suspended / Inactive:* System rejects registration with 403 TENANT_SUSPENDED or 403 TENANT_INACTIVE (or 403 TENANT_ARCHIVED for archived universities).
- *University Mismatch:* If user submits an explicit universityId conflicting with the email's domain-matched university, system rejects with 400 UNIVERSITY_MISMATCH.
- *Mailer Down:* The system records a development code in the audit log and returns emailDelivered: false instead of failing.

Postconditions: The account exists but cannot log in until the email OTP is verified.

FR-002: Email OTP Ownership Verification

Priority: High

Actor: Unauthenticated Registrant.

Preconditions: User has registered via FR-001 and possesses an unexpired, unconsumed OTP.

Description: Verifies user's ownership of the submitted email address via a 6-digit one-time code.

Main Flow:

1. User enters 6-digit code on frontend and submits POST /api/auth/verify-email.
2. System looks up user by email and retrieves active record from email_otps where purpose = 'signup' and consumed_at IS NULL.
3. System hashes input code with SHA-256 and verifies equality with stored code_hash.
4. System verifies expires_at > now() (10-minute TTL).
5. System marks OTP consumed_at = now() and updates user: email_verified = true, email_verified_at = now().
6. Writes user.email_verified entry to audit_logs.
7. Returns 200 OK allowing user to proceed to login.

Alternative / Exception Flows:

- *Invalid Code / Max Attempts Exceeded:* System increments attempts. If attempts >= max_attempts (5), the OTP is invalidated. Returns 400 OTP_INVALID.
- *Code Expired:* System rejects with 400 OTP_EXPIRED.
- *Resend Request:* User submits POST /api/auth/verify-email/resend. Enforces 60-second cooldown (OTP_RESEND_COOLDOWN_SECONDS) and maximum 5 resends per hour.

Postconditions: User's account is email-verified and unlocked for authentication.

FR-003: Login and Session Issuance

Priority: High

Actor: Registered User (Student, Merchant, Organizer, Campus Admin, Super Admin).

Preconditions: User account exists, is email-verified (email_verified = true), and is not suspended.

Description: Authenticates credentials and issues short-lived JWT access token and HttpOnly refresh token cookie.

Main Flow:

1. User submits email and password to POST /api/auth/login. Endpoint is rate-limited (max 8 requests per minute per IP/email).
2. System retrieves user and validates password hash using bcrypt.compare.
3. Checks suspended == false and email_verified == true.
4. If user is not super_admin, checks that user's university status is 'active'.
5. Generates 15-minute JWT access token signed with JWT_ACCESS_SECRET.
6. Generates 64-character random refresh token and unique family_id UUID.
7. Inserts SHA-256 hash of refresh token into refresh_tokens table with 30-day expiration.
8. Sets HttpOnly, SameSite=Lax, Path=/api/auth cookie named ch_refresh.

9. Generates 48-character random CSRF token, sets non-HttpOnly cookie ch_csrf, and returns access token and CSRF token in JSON body.

Alternative / Exception Flows:

- *Invalid Credentials:* Returns generic 401 Unauthorized without revealing whether email exists.
- *Unverified Email:* Returns 403 EMAIL_NOT_VERIFIED.
- *Suspended Account:* Returns 403 ACCOUNT_SUSPENDED.
- *Suspended Institution:* Returns 403 TENANT_SUSPENDED.

Postconditions: User authenticated; frontend stores access token in memory and CSRF token in state.

FR-004: Session Continuity and Refresh Token Rotation

Priority: High

Actor: Authenticated Client.

Preconditions: Client holds valid, unrevoked ch_refresh cookie and matching X-CSRF-Token header.

Description: Rotates refresh token, issues fresh access token, and detects token reuse attacks.

Main Flow:

1. Client sends POST /api/auth/refresh with ch_refresh cookie and X-CSRF-Token header.
2. System compares header CSRF token with ch_csrf cookie via crypto.timingSafeEqual.
3. Hashes refresh token with SHA-256 and retrieves record from refresh_tokens.
4. Validates token is not expired and revoked == false.
5. Marks current token as revoked = true.
6. Generates new refresh token under the same family_id.
7. Stores new token hash and sets updated ch_refresh cookie and new ch_csrf cookie.
8. Returns fresh 15-minute JWT access token.

Alternative / Exception Flows:

- *CSRF Mismatch:* Throws 403 CSRF_DETECTED.
- *Token Reuse Attack:* If presented token has revoked == true, system marks ALL tokens sharing that family_id as revoked, clears cookies, and returns 401 Refresh token reuse detected — all sessions revoked.

Postconditions: Active session maintained seamlessly; compromised tokens immediately neutralized.

FR-005: Password Reset via Emailed Token

Priority: Medium

Actor: User with Forgotten Password.

Preconditions: User has access to their registered email address.

Description: Allows a user to reset a forgotten password without revealing whether an account exists. Reset tokens are stored only as hashes.

Main Flow:

1. User submits email via POST /api/auth/forgot-password.
2. If account exists:
   - Generates 32-byte cryptographically secure random token (crypto.randomBytes(32).toString('hex')).

- Stores SHA-256 hash in users.password_reset_token with 1-hour expiration (password_reset_expires = now() + interval '1 hour').
- Sends email via mailer.sendMail with action link: https://<domain>/reset-password?token=<raw_token>.

3. System always responds with 200 OK and message: "If that email is registered, a password reset link has been sent" (prevents account enumeration).
4. User clicks link and submits new password via POST /api/auth/reset-password.
5. System hashes token from request, locates user where password_reset_expires > now().
6. Hashes new password with Bcrypt and updates password_hash.
7. Clears password_reset_token and password_reset_expires.
8. Revokes all active sessions in refresh_tokens for that user.

Alternative / Exception Flows:

- *Expired / Tampered Token:* Returns 400 Invalid or expired reset token.

Postconditions: Password updated; old sessions invalidated across all devices.

FR-006: User Logout

Priority: Medium

Actor: Authenticated User.

Preconditions: Active session cookies exist.

Description: Ends the current session by revoking its refresh token.

Main Flow:

1. User clicks "Sign Out" triggering POST /api/auth/logout.
2. System validates CSRF token.
3. Locates refresh token from cookie and sets revoked = true in refresh_tokens.
4. Clears ch_refresh and ch_csrf cookies using empty values and past expiration.
5. Returns 200 OK.

Postconditions: The session ends and the refresh token cannot be used again.

5.2 User Profiles and Account Management

FR-007: Role-Adaptive Profile View and Customization

Priority: Medium

Actor: Authenticated User (Any Role).

Preconditions: Valid access token.

Description: Displays profile header, identity badges, and allows updating editable fields while preventing modification of protected institutional fields.

Main Flow:

1. User requests GET /api/users/me to load profile data.
2. System returns user record joined with universities (name, domain).
3. User submits updates via PATCH /api/users/me.

4. Allows editing: name, phone, bio, department, year, avatarUrl, coverImageUrl, privacySettings, notificationPrefs.
5. Rejects any attempt to modify protected attributes (role, verified, verification_method, email_verified, suspended, university_id, id) with 403 PROTECTED_FIELD.
6. Writes user.profile_update to audit_logs.

Postconditions: Profile updated; audit log entry recorded.

FR-008: Password Change with Current Password Verification

Priority: Medium

Actor: Authenticated User.

Preconditions: User knows their current password.

Description: Allows user to change their password while logged in, requiring verification of existing password.

Main Flow:

1. User submits currentPassword and newPassword (min 8 chars) to POST /api/users/me/password.
2. System retrieves current password_hash and verifies currentPassword via bcrypt.compare.
3. Hashes newPassword and updates password_hash.
4. Revokes all refresh tokens in refresh_tokens for this user (logs out other devices).
5. Writes user.password_change to audit_logs.

Alternative / Exception Flows:

- *Incorrect Current Password:* Returns 400 INVALID_CURRENT_PASSWORD.

Postconditions: Password changed; existing sessions on other devices invalidated.

FR-009: Role-Specific Profile Analytics and Statistics

Priority: Low

Actor: Authenticated User.

Preconditions: User is authenticated.

Description: Provides computed operational statistics tailored to the user's role via GET /api/users/me/stats.

Role Mappings:

- **Student:** Returns orders count, rental bookings count, active P2P listings count, active/used tickets count, favorites count, and reviews given count.
- **Merchant:** Returns store status, total products, active products, total orders, completed orders, total revenue, average store rating, and review count.
- **Organizer:** Returns club status, total events, upcoming events, completed events, tickets sold, and total attendees admitted.
- **Admin / Super Admin:** Returns users managed, active merchants, registered clubs, open moderation reports, total completed orders, and platform revenue.

Postconditions: Returns computed real-time metrics for profile dashboard display.

FR-010: Unified Personal Activity Feed

Priority: Low

Actor: Authenticated User.

Preconditions: User is authenticated.

Description: Aggregates recent cross-module activity into a unified chronological feed via GET /api/users/me/activity.

Behavior:

- Combines recent orders, rental bookings, P2P listings, and event tickets into a single chronological array sorted by date descending (limit 15).
- Each item provides type, title, subtitle, status, date, and actionable deep link.

Postconditions: Displays activity timeline on profile page.

FR-011: Public Profile with Cross-Campus Privacy Masking

Priority: Medium

Actor: Any User (Public or Authenticated).

Preconditions: Target user account is not suspended.

Description: Exposes public seller/organizer identity via GET /api/users/:id/public while respecting privacy toggles and enforcing cross-campus data masking.

Privacy Enforcement:

- Respects user's privacy_settings: email shown only if showEmail == true; phone shown only if showPhone == true.
- **Cross-Campus Masking:** If viewer is unauthenticated or belongs to a different university than the target student:
- department and year are masked as null.
- Activity showcases are hidden unless user is on the same campus or viewer is super_admin.
- Showcases active store and rated products for merchants, upcoming approved events for organizers, and active listings/rentals for students.

Postconditions: Public profile safely rendered without privacy leaks.

5.3 Merchant Marketplace and Product Catalog

FR-012: Merchant Application and Agreement Onboarding

Priority: High

Actor: Student wishing to sell commercial goods.

Preconditions: Account is email-verified and campus-verified.

Description: Allows user to submit a formal merchant application agreeing to platform commission terms.

Main Flow:

1. User submits POST /api/merchants/apply with businessName, category, contactInfo, and supporting documents.
2. System records active terms agreement in application_agreements referencing current agreement_versions (recording 5.0% commission obligation).
3. Creates record in merchant_applications with status 'submitted'.
4. Notifies campus administrators via in-app notification.

5. Admin reviews via PATCH /api/merchants/:id/decision.
6. Upon approval, system upgrades user's role to 'merchant' and creates an initial store record in stores with status 'active'.

Postconditions: User transitioned to merchant role with managed store.

FR-013: Store Profile Management

Priority: Medium

Actor: Campus Merchant.

Preconditions: Merchant has an approved store.

Description: Allows merchant to customize store branding, contact details, and banner images via PATCH /api/merchants/stores/mine.

- **Attributes:** Store name, description, category, public email, public phone, logo URL, and cover image URL.

FR-014: Product Inventory Management

Priority: High

Actor: Campus Merchant.

Preconditions: Merchant owns an active store.

Description: Allows creating, updating, and removing products via /api/products.

Validation Rules:

- Title (min 3 chars), description, price (positive decimal), stock level (integer >= 0), category ID, image URLs array.
- System automatically transitions product status between 'active' and 'out_of_stock' based on stock quantity.
- Soft deletion sets status to 'removed', preserving order history integrity.

FR-015: Marketplace Browsing and Multi-Faceted Search

Priority: High

Actor: Any User / Student.

Preconditions: University tenant resolved.

Description: Allows browsing active products scoped to the user's university.

- **Features:** Text search across product titles/descriptions, category filtering, min/max price bounds, and pagination (default 20 items per page).

FR-016: Favorite Items Management

Priority: Low

Actor: Authenticated Student.

Preconditions: Product is active.

Description: Toggles bookmarking of products via POST /api/products/:id/favorite and DELETE /api/products/:id/favorite, persisting entries in favorites table.

5.4 Order Processing and Physical Handover Lifecycle

FR-017: Order Placement and Atomic Stock Hold

Priority: High

Actor: Verified Student.

Preconditions: Student is verified (verified == true); product stock is sufficient.

Description: Initiates purchase of one or more items from a single merchant store within an atomic database transaction.

Main Flow:

1. Student submits basket items and paymentMethod: 'manual' to POST /api/orders.
2. Database transaction begins:
   - Locks product rows with SELECT ... FOR UPDATE.
   - Validates all products belong to the same merchant store and same university.
   - Verifies requested quantities against current stock.
   - Atomically decrements stock: UPDATE products SET stock = stock - $qty.
   - If stock reaches 0, sets product status to 'out_of_stock'.
   - Inserts record into orders with status 'pending_payment'.
   - Inserts line items into order_items.
   - Inserts initial status record into order_status_history.
3. Initiates payment record via payments.initiatePayment with provider = 'manual', transaction_type = 'order', and status = 'pending'.
4. Returns 201 Created with order details and payment ID.

Alternative / Exception Flows:

- *Insufficient Stock:* Transaction rolls back; returns 409 OUT_OF_STOCK identifying the scarce item.
- *Multi-Store Cart:* Rejects with 400 All items in one order must be from the same store.

Postconditions: Order created in pending_payment; product inventory locked.

FR-018: Two-Party Order Fulfillment Confirmation

Priority: High

Actor: Merchant and Buyer (Student).

Preconditions: The order exists and has not been completed or cancelled. The first step starts from pending_payment; later steps start from confirmed or ready.

Description: Enforces two-party physical verification: merchant confirms payment receipt, and buyer confirms physical goods handover.

Fulfillment Rules:

1. **Merchant Confirms Payment:** When buyer pays merchant directly (via CBE/Telebirr) and merchant approves proof or confirms receipt, payment moves to 'confirmed'. Order status automatically updates to 'confirmed' with merchant_confirmed_at = now(). The 5.0% commission entry is recorded in the same database transaction (FR-038).

2. **Merchant Prepares Order:** Merchant moves order from confirmed to ready (“Ready for Pickup”) via PATCH /api/orders/:id/status.
3. **Buyer Confirms Physical Receipt:** When student inspects and collects physical material, student invokes POST /api/orders/:id/confirm-receipt.
4. System records buyer_confirmed_at = now() and advances order status to ‘completed’.
5. Both parties receive confirmation notifications.

Postconditions: Order finalized as ‘completed’; the 5.0% commission entry was already recorded in the ledger when the payment was confirmed (FR-038).

FR-019: Order Cancellation and Inventory Restocking

Priority: Medium

Actor: Buyer, Merchant, or Admin.

Preconditions: Order has not reached ‘completed’ or ‘refunded’.

Description: Handles buyer cancellation requests and immediate merchant/admin cancellations with atomic stock restoration.

Cancellation Flow:

- If unpaid (pending_payment), buyer can cancel immediately.
- If paid (confirmed or ready), buyer submits cancellation request setting status to ‘cancellation_requested’. Merchant or Admin reviews and approves cancellation.
- Upon cancellation:
- Status set to ‘cancelled’.
- Database transaction invokes orderStatus.releaseStock, restoring quantities to products.stock.
- If payment was initiated, payment record marked ‘failed’.

Postconditions: Order cancelled; product stock released back to marketplace.

FR-020: Order History, Detail Tracking and Audit Logs

Priority: Medium

Actor: Buyer, Merchant, Campus Admin.

Preconditions: User is authenticated.

Description: Shows order details, itemized breakdowns, store information, and full status progression timeline via GET /api/orders/:id and GET /api/orders/:id/history.

5.5 Peer-to-Peer (P2P) Student Marketplace

FR-021: Create and Manage P2P Listings

Priority: Medium

Actor: Verified Student.

Preconditions: Feature flag p2p is enabled for university.

Description: Allows students to post used items (textbooks, electronics, lab coats) via POST /api/listings.

- **Fields:** Title, description, price, condition (new, like_new, good, fair), category ID, and uploaded image URLs. Status defaults to ‘active’.

FR-022: P2P Buyer Buy/Contact Requests

Priority: Medium

Actor: Student Buyer.

Preconditions: Listing is active and belongs to same campus.

Description: Allows interested buyer to submit a buy request with offer message via POST /api/listings/:id/requests. System notifies seller with actionable notification link.

5.6 Borrowing and Equipment Rental System

FR-023: List Equipment for Rent

Priority: High

Actor: Verified Student or Merchant.

Preconditions: Feature flag rentals is enabled.

Description: Lists equipment (scientific calculators, lab kits, cameras) for rent via POST /api/rentals.

- **Attributes:** Title, description, category, price_per_day, refundable deposit, availability date range (available_from to available_to), image URLs, and optional tiered late-fee policy.

FR-024: Book Rental Item and Calculate Deposit

Priority: High

Actor: Student Borrower.

Preconditions: Item is active and available for requested date range.

Description: Creates rental reservation via POST /api/rentals/:id/bookings.

- **Calculation:** Total amount = (days * price_per_day) + deposit. Status set to ‘pending_payment’.

FR-025: Rental Two-Party Pickup and Return Confirmation

Priority: High

Actor: Item Owner and Borrower.

Preconditions: Rental booking payment confirmed.

Description: Lifecycle tracking for physical equipment exchange:

1. **Direct Payment:** Borrower pays owner directly. Owner confirms payment receipt (owner_confirmed_at = now()), setting booking status to ‘confirmed’.
2. **Physical Handover (Pickup):** Borrower inspects item and confirms pickup via POST /api/rentals/bookings/:id/pickup. Sets picked_up_at = now() and status to ‘active’.
3. **Return & Inspection:** Borrower returns item. Owner inspects physical condition and confirms return via POST /api/rentals/bookings/:id/return. Sets returned_at = now(), status to ‘completed’, and resolves deposit status.
4. Platform commission is **0%** (free student sharing utility).

FR-026: Tiered Late-Fee Escalation and Rental Disputes

Priority: Medium

Actor: Item Owner, Borrower, Campus Admin.

Preconditions: Rental item overdue or returned damaged.

Description:

- Automated late fee calculation based on days past end_date.
- Owner or borrower can file formal dispute via POST /api/rentals/bookings/:id/dispute.
- The dispute enters the Campus Admin queue with evidence photos. The admin records a resolution (for example deduct the deposit, issue a warning, or suspend the account).

5.7 Campus Event Management

FR-027: Student Club and Organizer Onboarding

Priority: High

Actor: Student Club Leader.

Preconditions: Campus-verified student.

Description: Submits club application via POST /api/organizers/apply. Upon Campus Admin review and approval, user role upgraded to ‘organizer’ and club record created in clubs.

FR-028: Event Creation with Recurring Schedule

Priority: High

Actor: Event Organizer.

Preconditions: Organizer belongs to active club.

Description: Creates event draft via POST /api/events.

- **Fields:** Title, description, category, poster image URL, date_time, end_time, venue name, venue address, total capacity, contact info, and recurrence_rule (e.g., ‘none’, ‘weekly’). Status initializes as ‘draft’.

FR-029: Event Administrative Review and Approval

Priority: High

Actor: Campus Admin.

Preconditions: Organizer submitted event via POST /api/events/:id/submit.

Description: Admin reviews event content, timing, and venue safety.

- Approve: moves status to ‘approved’ or ‘published’, opening ticket sales.
- Reject: records reject_reason and notifies organizer to revise.

5.8 Digital Ticketing, Cryptographic QR and Entry Validation

FR-030: Multi-Tier Ticket Configuration

Priority: High

Actor: Event Organizer.

Preconditions: Event created.

Description: Configures ticket tiers in ticket_types: name (e.g., “Regular”, “VIP”, “Free Student Entry”), price, total quantity quota, purchase limit per student, and sales time window.

FR-031: Ticket Purchase and Manual Payment Proof Submission

Priority: High

Actor: Student Attendee.

Preconditions: Event published, ticket type active, quotas available.

Description:

1. Student selects ticket tier and quantity via POST /api/events/:id/tickets.
2. For paid tickets:
   - Creates ticket records in tickets with status 'pending_payment'.
   - Creates payment record in payments with provider = 'manual', transaction_type = 'ticket', and university receiving accounts in payee_accounts.
   - Generates human ticket number: GIB-XXXXXXXX (8 uppercase hex digits).
3. Student transfers payment to the campus receiving account (CBE / Telebirr).
4. Student uploads screenshot and reference number via POST /api/payments/:id/proof.
5. Payment moves to status 'under_review'.
6. For free tickets (price == 0): ticket immediately activated with status 'active'.

FR-032: Campus Admin Ticket Payment Approval and QR Activation

Priority: High

Actor: Campus Administrator (or Super Administrator).

Preconditions: Payment in under_review for transaction type 'ticket'.

Description:

- **Access rule:** Only a Campus Administrator of the same university (or a Super Administrator) can approve ticket payment proofs. Students and organizers cannot.
- Admin reviews transaction reference and receipt screenshot via POST /api/payments/:id/approve.
- Transaction finalizer updates payment to 'confirmed'.
- Updates all associated tickets to status = 'active'.
- Platform records **2.5% platform commission** in commissions table.
- Generates HMAC-SHA256 signature for ticket QR.
- Organizer receives 97.5% net amount via manual payout process (organizer_payouts).

FR-033: Cryptographic QR Generation and Key Rotation

Priority: High

Actor: System Service (utils/qr.js).

Preconditions: Ticket status is 'active'.

Description:

- Constructs QR payload string: <ticket_id>.<signature>.
- Signature computed as crypto.createHmac('sha256', TICKET_QR_SECRET).update(ticketId).digest('hex').
- Supports key rotation: validator checks primary TICKET_QR_SECRET, falling back to TICKET_QR_SECRET_PREVIOUS.

- QR image rendered as PNG Data-URL for display in “My Tickets” view.

FR-034: Central Ticket Identifier Normalization

Priority: High

Actor: Event Staff / Organizer / Camera Scanner.

Preconditions: Scanner or staff inputs ticket data.

Description: ticketValidation.js parses and classifies any input variant:

1. Verification URL (https://.../tickets/verify/<uuid>.<hmac>).
2. Signed QR string (<uuid>.<hmac>).
3. Bare UUID (<uuid>).
4. Ticket Number (GIB-F7A79799, gib-f7a79799, F7A79799, GIBF7A79799).

FR-035: One-Time Admission Entry Check-In

Priority: High

Actor: Event Staff / Organizer.

Preconditions: Ticket is cryptographically valid and belongs to the event.

Description:

1. Staff scans QR or enters ticket number at entry gate.
2. System executes atomic check:
   - If status == ‘active’ and used_at IS NULL:
   - Updates ticket: status = ‘used’, used_at = now(), used_by = staff.id.
   - Inserts record into ticket_checkins with staff_id, checked_in_at, and device_info.
   - Returns green **VALID — ADMISSION GRANTED** response with attendee name, tier, and timestamp.
   - If status == ‘used’:
   - Rejects entry with red **ALREADY USED** response, displaying original check-in timestamp and staff name.
   - If payment rejected or cancelled:
   - Displays cancellation notice and denies entry.

FR-036: Admission Reversal and Attendee Roster

Priority: Medium

Actor: Event Organizer, Campus Admin, or Super Admin.

Preconditions: Ticket was erroneously checked in.

Description:

- Allows staff to reverse an accidental scan via POST /api/tickets/:id/reverse-checkin.
- Resets ticket to status = ‘active’, used_at = NULL.
- Logs reversal record in ticket_checkins (is_reversal = true, reversed_by, reversed_at, notes).
- Provides searchable attendee roster (GET /api/events/:id/attendees) showing real-time entry counts.

5.9 Payment Processing and Commission Management

FR-037: Manual Payment System and Receiving Accounts

Priority: High

Actor: Platform Administration.

Preconditions: University tenant established.

Description:

- System maintains official institutional receiving accounts in payout_accounts with is_campus_hub = true (e.g., Commercial Bank of Ethiopia, Telebirr).
- Configures account number, account name, and payment instructions.

FR-038: Platform Commission Rules and Ledgering

Priority: High

Actor: System Service (services/commissions.js).

Preconditions: Payment confirmation event occurs.

Description:

- **Material Sales (Orders):** Platform commission = **5.0%** (COMMISSION_RATE_ORDER = 0.05). Billed to merchant; merchant pays platform.
- **Event Tickets:** Platform commission = **2.5%** (COMMISSION_RATE_TICKET = 0.025). Deducted from gross ticket sales before organizer payout (97.5%).
- **Equipment Rentals:** Platform commission = **0%** (COMMISSION_RATE_BOOKING = 0.00). Peer-to-peer student service.
- **Other Transaction Types:** Payment types without a dedicated rate (currently in-app P2P orders, p2p_order) use the fallback commission rate of the university fee schedule (default 5.0%, COMMISSION_RATE) and are billed to the seller. Promotion payments (FR-046) carry no seller commission. Per-type rates for orders, tickets and rentals are not overridden by fee schedules.
- Records individual commission row in commissions table linked to payment_id and payee_id.

FR-039: Merchant Commission Proof Submission and Settle Queue

Priority: High

Actor: Campus Merchant and Campus Admin.

Preconditions: Merchant has accumulated due commissions.

Description:

1. Merchant views outstanding commission balance via GET /api/commissions/mine.
2. Merchant transfers commission amount to the Gibiyachin official receiving account.
3. Merchant submits screenshot and reference via POST /api/commissions/submit-payment.
4. Campus Admin reviews submitted proofs via GET /api/commissions/admin/payments.
5. Admin verifies payment via POST /api/commissions/admin/payments/:id/verify.
6. **Whole-Entry Chronological Settlement:** System allocates verified amount chronologically to oldest ‘due’ commission entries. Partial surpluses are strictly rejected to prevent accounting ambiguity.
7. Marked commissions update to status = ‘settled’.

5.10 Actionable Notifications and Campus Verification

FR-040: Actionable Notification Delivery and Deep-Linking

Priority: Medium

Actor: All Users.

Preconditions: System event triggered.

Description:

- Writes notification row to notifications with target_route, target_entity_type, and target_entity_id.
- When user clicks notification:

1. Sends asynchronous PATCH /api/notifications/:id/read.
2. Client validates destination route against allowlist ALLOWED_ROUTE_PREFIXES.
3. Navigates directly to the exact target resource (e.g., /orders?orderId=..., /tickets?ticketId=...).

FR-041: Campus ID Document Manual Verification

Priority: Medium

Actor: Student and Campus Admin.

Preconditions: Student holds non-institutional email (e.g., Gmail).

Description:

- Student uploads student ID card or enrollment slip via POST /api/verifications/submit.
- File validated via magic bytes and stored securely in private_uploads/proofs/.
- Campus Admin reviews pending queue via GET /api/admin/verifications/requests.
- Upon approval, user's account updated: verified = true, verification_method = 'manual_document'.

5.11 Platform Governance and Multi-Tenancy

FR-042: Multi-Campus Tenant Management

Priority: High

Actor: Super Administrator.

Preconditions: Super Admin authentication.

Description:

- Super Admin onboards universities via POST /api/super/universities (specifying name, slug, domains, city, region, institution type, and feature flags).
- Creates initial Campus Administrator accounts.
- Monitors system-wide health, orders, and security audit logs.

5.12 Extended Platform Capabilities

The following requirements describe capabilities that were added to this SRS in version 2.1. Earlier versions did not specify them.

FR-043: Ticket Waitlist for Sold-Out Tiers

Priority: Low

Actor: Student Attendee.

Preconditions: The ticket type is sold out and the event is published.

Description:

- A student joins the waitlist via POST /api/events/:eventId/waitlist with a ticketTypeId (student role only); the entry is stored in waitlist_entries with status waiting (other states defined in the schema: notified, claimed, expired). Joining is rejected while the tier still has availability, so the waitlist cannot be used to skip the line.
- Student lists personal waitlist entries via GET /api/events/waitlist/mine.
- When a spot is freed (for example by ticket cancellation or expiry), the system selects the oldest waiting entry for that tier (first come, first served), sets it to notified with notified_at, and sends a ticket_update notification. The spot is not reserved for the notified student.

Postconditions: Waitlisted students are notified in arrival order when capacity reappears.

FR-044: Reviews and Ratings

Priority: Medium

Actor: Authenticated User.

Preconditions: The user has a qualifying completed transaction with the target.

Description:

- User submits POST /api/reviews with targetType (product, store, listing or user), targetId, an integer rating from 1 to 5, and an optional comment of up to 2000 characters; reviews are listed via GET /api/reviews.
- Eligibility: a product or store may be reviewed only by a buyer with a completed order containing that product or from that store; a P2P listing only by a buyer whose purchase request for it was accepted; a user (rental counterpart, lender or borrower) only after a completed booking between the two users.
- Reviews without a qualifying completed transaction are rejected with 403.

Postconditions: Ratings reflect only verified completed transactions.

FR-045: Content Reporting and Moderation

Priority: Medium

Actor: Any Authenticated User; Campus Administrator.

Preconditions: The reported entity belongs to the reporter’s campus.

Description:

- User reports content via POST /api/reports with targetType (product, listing, user, store or event), a reason (minimum 2 characters) and optional details; administrators of the reporter’s university are notified with a deep link to the report.
- Administrator reviews reports via GET /api/admin/reports and closes them via PATCH /api/admin/reports/:id/resolve with a resolution (minimum 2 characters), an action (remove_listing, warn, suspend, refund or dismiss) and optional notes.
- Administrator removes or restores a product or listing via PATCH /api/admin/moderate with targetType (product or listing) and action of remove or restore, which sets the status to removed or active within the administrator’s university and writes a content.moderate entry to audit_logs.

Postconditions: Abusive content is removed from public view and the decision is audited.

FR-046: Featured Product Promotion

Priority: Low

Actor: Campus Merchant.

Preconditions: The merchant owns an active product.

Description:

- Merchant requests promotion via POST /api/products/:id/promote with days (1 to 30, default 7).
- The amount is the university's featured_rate_per_day multiplied by days (default 20 ETB per day when no fee schedule exists), and a payment of type promotion is initiated.
- On payment confirmation, products.featured_until is extended by the purchased days, counted from the later of the current time and the existing featured_until. No seller commission is recorded for promotion payments.

Postconditions: The product is ranked as featured until featured_until.

FR-047: Offline Scan Queue and Batch Check-In

Priority: Medium

Actor: Organizer, Campus Administrator or Super Administrator.

Preconditions: The user holds a role permitted to scan tickets.

Description:

- When connectivity is lost, the scanner screen queues scans locally together with their scan times.
- On reconnection the queue is submitted via POST /api/tickets/verify-batch with up to 200 entries, each carrying ticketId, qrToken or ticketNumber and an optional scannedAt, plus an optional eventId. The endpoint is rate limited to 20 requests per minute per user.
- The server orders entries by scannedAt and applies the same validation and check-in rules as FR-035 to each entry, returning one result per scan (admitted, already used, invalid).

Postconditions: Offline admissions are reconciled in scan order with the standard audit trail.

FR-048: CSV Data Export

Priority: Low

Actor: Campus Administrator; Campus Merchant.

Preconditions: The user is authenticated with the corresponding role.

Description:

- Campus administrators export data via GET /api/admin/reports/export?type=orders|transactions|users, returned as text/csv with a download file name; an unknown type is rejected with 400.
- Store orders are exported via GET /api/merchants/:storeId/orders/export, allowed for the store owner, Campus Administrators and Super Administrators.

Postconditions: A CSV file scoped to the user's campus or store is downloaded.

FR-049: Recurring Event Occurrences

Priority: Low

Actor: Event Organizer.

Preconditions: The template event is in status published.

Description:

- Organizer calls POST /api/events/:id/generate-occurrences with count (1 to 12) and intervalDays (1 to 90, default 7).
- If the template's recurrence_rule is none, it is set to weekly. The system creates count child events with parent_event_id set to the template, the same club, title, venue and capacity, a start time shifted by intervalDays multiplied by the occurrence number, status published and recurrence_rule none.
- Occurrences are listed via GET /api/events/:id/occurrences. Attempting to use a non-published event as a template is rejected with 409.

Postconditions: Recurring sessions of one event are published in a single operation.

FR-050: In-App P2P Order Payment

Priority: Medium

Actor: Student Buyer; Student Seller.

Preconditions: The seller has accepted the buyer's request, creating a p2p_orders record in status pending_payment, and the seller has an active payout account.

Description:

- Seller accepts or declines a request via PATCH /api/listings/requests/:requestId; buyer and seller may read the resulting order via GET /api/listings/requests/:requestId/order.
- Buyer pays via POST /api/listings/requests/:requestId/pay with an optional paymentMethod (online or manual). The call returns 404 if no accepted order exists yet, 400 NO_PAYOUT_ACCOUNT if the seller has not registered a payout account, and 409 if the order is no longer pending_payment; only the buyer or seller may access the order. The created payment has type p2p_order, the order amount, and the seller as payee.
- On payment confirmation the order becomes paid, the listing becomes sold, and the seller receives a notification to arrange the handover. Commission follows FR-038 (other transaction types).

Postconditions: The P2P item is sold and the seller is notified to hand it over.

FR-051: Fee Schedules and Agreement Versioning

Priority: Medium

Actor: Super Administrator.

Preconditions: Super Admin authentication.

Description:

- Super Admin creates and lists university fee schedules via POST and GET /api/super/fee-schedules (commission_rate, featured_rate_per_day, effective_from). The latest effective schedule supplies the fallback commission rate (FR-038) and the promotion rate (FR-046); when none exists, environment defaults apply.
- Super Admin publishes and lists agreement versions (merchant_terms, organizer_terms) via POST and GET /api/super/terms; applicants agree to the active version, which is recorded in application_agreements (FR-012, FR-027).

Postconditions: Campus-specific fees and legal terms are versioned and auditable.

FR-052: Account Deactivation and Deletion

Priority: Medium

Actor: Campus Administrator or Super Administrator.

Preconditions: The target account is manageable by the administrator.

Description:

- Administrator deactivates an account via POST /api/admin/users/:id/deactivate with an optional reason; the account is marked suspended, blocked at login and on every authenticated request, and its historical records remain. A second request returns 409 ALREADY_SUSPENDED.
- Administrator inspects related records via GET /api/admin/users/:id/dependencies, which returns record counts and a canDelete flag.
- Permanent deletion via DELETE /api/admin/users/:id is allowed only when no dependent records exist; otherwise the request fails with DEPENDENCIES_EXIST. Deactivations are written to audit_logs as ADMIN_DEACTIVATED_MERCHANT, ADMIN_DEACTIVATED_ORGANIZER or ADMIN_DEACTIVATED_USER.

Postconditions: The account can no longer authenticate, and the action is audited.

6. Non-Functional Requirements (NFRs)

This section lists the non-functional requirements. The numeric limits (for example response times and availability) are targets set by the project team; they are not test results. Section 15.2 shows, for each requirement, whether any measurement or test exists. Where nothing has been measured, the requirement is a target that still needs validation.

6.1 Performance Requirements

- **NFR-001 (Read Endpoint Latency):** Under normal pilot load (up to 200 concurrent active campus users), read API endpoints (marketplace catalog, public profile, events feed) SHALL maintain a 95th-percentile (P95) response time under 350 ms.
- **NFR-002 (Transaction Execution Time):** Stateful mutation transactions (placing orders, booking rentals, issuing tickets) SHALL complete within 500 ms at P95, bounded by row-level locking.
- **NFR-003 (Database Query Optimization):** All foreign-key lookups, status filters, and user activity queries SHALL utilize indexed columns. Query execution plans (EXPLAIN ANALYZE) SHALL avoid sequential table scans on core transactional tables (orders, tickets, commissions).
- **NFR-004 (Frontend Bundle Efficiency):** The client-side distribution bundle generated by Vite SHALL NOT exceed 500 kB (compressed Gzip/Brotli) for initial route loading.

6.2 Security and Data Protection Requirements

- **NFR-005 (Transport Layer Encryption):** All external and client-server HTTP communications MUST enforce TLS 1.2 or higher with HSTS enabled.
- **NFR-006 (Password Storage):** User passwords MUST be stored exclusively as one-way cryptographic salted hashes generated using Bcrypt with a minimum work factor of 10. Plaintext passwords SHALL NEVER be logged or stored.
- **NFR-007 (Session Storage & CSRF):** Refresh tokens MUST be transmitted exclusively via HttpOnly, SameSite=Lax cookies restricted to /api/auth (the Secure attribute is enabled when NODE_ENV=production). Every state-changing refresh and logout request MUST validate an anti-CSRF token via constant-time comparison (crypto.timingSafeEqual).
- **NFR-008 (Cryptographic QR Anti-Tampering):** Event ticket QR codes MUST contain HMAC-SHA256 signatures generated from high-entropy platform secrets. Verification MUST reject any payload that fails timing-safe signature comparison.
- **NFR-009 (Multi-Tenant Data Isolation):** The application layer MUST enforce that no user of one university can view or change data that belongs to another university. A cross-tenant request MUST be rejected; a conflicting tenant header is rejected with 403 Forbidden (TENANT_MISMATCH).
- **NFR-010 (SQL Injection Prevention):** All database operations MUST utilize parameterized queries via the pg client library. String concatenation of user inputs into SQL statements is strictly prohibited.
- **NFR-011 (Image Sanitization & Polyglot Prevention):** All user-uploaded raster images MUST be processed, metadata-stripped, auto-rotated, and re-encoded through Sharp (libvips) before persistence. Uploads that do not match magic-byte signatures MUST be rejected with HTTP 400.

6.3 Reliability and Availability Requirements

- **NFR-012 (Platform Uptime):** The system SHALL achieve an operational availability of at least 99.0% during standard campus operating hours (07:00 to 22:00 EAT), excluding scheduled maintenance windows.
- **NFR-013 (Financial & Inventory Atomic Consistency):** In the event of network disruption, payment failure, or server error during order placement, rental booking, or ticket purchasing, database state MUST rollback atomically (ROLLBACK), preventing orphan financial records or unallocated stock deductions.
- **NFR-014 (Audit Trail Immutability):** High-risk administrative, financial, and entry check-in actions MUST be recorded in append-only audit_logs and ticket_checkins tables. No application API endpoint updates or deletes rows in these tables (confirmed by code search); check-in reversals are recorded as new rows with is_reversal = true. Database-level enforcement (privilege revocation or triggers) is listed as a hardening item in Section 14.3.

6.4 Usability and Accessibility Requirements

- **NFR-015 (Responsive Mobile Usability):** The frontend user interface MUST be fully responsive and functional across screen viewports from 320 px (compact mobile smartphones) to 2560 px (wide desktop monitors).
- **NFR-016 (Visual Clarity & Status Indication):** Order fulfillment, payment review, verification status, and ticket admission states MUST use universally recognizable color codings (Green = Approved/Valid, Amber = Pending Review/Hold, Red = Rejected/Cancelled/Void) accompanied by clear textual status labels.
- **NFR-017 (Scanner Responsiveness):** The camera-based QR scanner component (html5-qrcode) SHALL acquire, decode, and transmit valid ticket QR payloads in under 1.5 seconds under ambient campus lighting conditions.

6.5 Maintainability and Scalability Requirements

- **NFR-018 (Modular Monolith Structure):** Backend modules MUST maintain low coupling and high cohesion. Domain logic MUST be isolated into discrete modules (auth, products, orders, tickets, commissions, verifications, super).
- **NFR-019 (Database Migration Versioning):** All schema modifications MUST be documented as ordered, idempotent SQL migration scripts capable of running cleanly from clean setup to current state. The npm script db:migrate:all executes schema.sql followed by migrations 003 to 021 in order (including 006 and 008) and was run from an empty database to produce the full 40-table schema.
- **NFR-020 (Horizontal Backend Scalability):** The backend application SHALL maintain stateless session handling (with session states delegated to JWT tokens and PostgreSQL persistence), allowing multiple Express.js instances to run concurrently behind a reverse proxy (e.g., NGINX). Deployment conditions: when more than one instance runs, REDIS_URL MUST be configured (otherwise rate limits are counted per process) and the /uploads and /private_uploads directories MUST reside on shared storage.

7. Database Design and Entity Relationship Specifications

Gibiyachin stores its data in PostgreSQL. The schema has 40 tables with foreign keys, indexes, and check constraints. Figure 3 shows the main relationships, and Section 7.1 lists every table. The dashed lines in Figure 3 are logical references: `payments.transaction_id` points to a different table depending on `payments.transaction_type`.

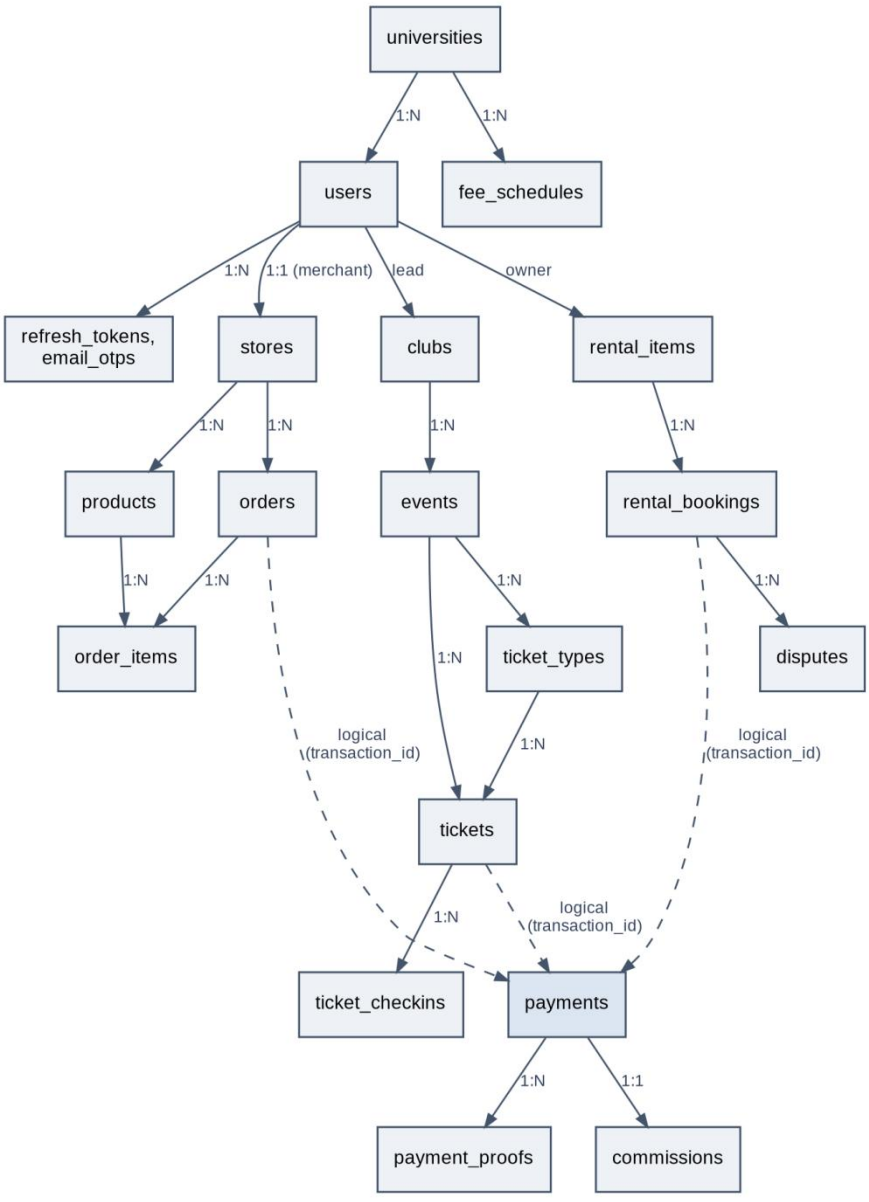


Figure 3: Main entity relationships (not all 40 tables are shown)

7.1 Database Entity Catalog (40 Tables)

Domain A: Multi-Tenancy and Identity Core (7 Tables)

1. **universities:** Institutional anchor. Columns: id (UUID PK), name, slug, domain, permitted_domains (text[]), status (active/suspended/archived), feature_flags (JSONB), isolation_mode, code, city, region, institution_type (public/private).
2. **users:** Core user profile. Columns: id (UUID PK), university_id (FK), name, email, phone, password_hash, role (enum: student, merchant, organizer, admin, super_admin), verified (bool), verification_method, student_id_doc_url, suspended (bool), avatar_url, cover_image_url, bio, department, year, privacy_settings (JSONB), notification_prefs (JSONB), email_verified (bool), email_verified_at.
3. **refresh_tokens:** Session token tracking. Columns: id (UUID PK), user_id (FK), token_hash (SHA-256), family_id (UUID), revoked (bool), expires_at, created_at.
4. **email_otps:** Email verification one-time passwords. Columns: id (UUID PK), user_id (FK), purpose, code_hash (SHA-256), expires_at, attempts, max_attempts, send_count, last_sent_at, consumed_at.
5. **verification_requests:** Campus identity document verification queue. Columns: id (UUID PK), university_id (FK), user_id (FK), document_type, document_url, student_note, status (pending/approved/rejected), admin_reviewer_id (FK), rejection_reason, reviewed_at.
6. **categories:** Product, rental, and listing taxonomy. Columns: id (UUID PK), name, parent_category_id (FK).
7. **favorites:** Student bookmarked items. Columns: user_id (FK), product_id (FK), created_at. Composite PK (user_id, product_id).

Domain B: Merchant Stores, Products and Orders (6 Tables)

1. **stores:** Merchant store identity. Columns: id (UUID PK), merchant_id (FK users), university_id (FK), name, logo_url, cover_image_url, category, public_email, public_phone, description, status (active/suspended).
2. **products:** Commercial catalog items. Columns: id (UUID PK), store_id (FK stores), category_id (FK), title, description, price, stock, images (text[]), status (active/out_of_stock/removed), featured_until.
3. **orders:** Merchant order fulfillment records. Columns: id (UUID PK), university_id (FK), buyer_id (FK users), store_id (FK stores), status (pending_payment/confirmed/ready/completed/cancelled/refunded/cancellation_requested), total_amount, merchant_confirmed_at, buyer_confirmed_at, cancel_reason.
4. **order_items:** Line items per order. Columns: id (UUID PK), order_id (FK orders), product_id (FK products), quantity, unit_price.
5. **order_status_history:** Audit timeline of order status transitions. Columns: id (UUID PK), order_id (FK orders), university_id (FK), from_status, to_status, changed_by (FK users), note, created_at.
6. **merchant_applications:** Merchant onboarding queue. Columns: id (UUID PK), user_id (FK), university_id (FK), business_name, category, contact_info, document_urls (text[]), status (submitted/approved/rejected), reviewed_by (FK), reviewed_at.

Domain C: Peer-to-Peer and Rental Sharing (5 Tables)

1. **p2p_listings:** Used student classifieds. Columns: id (UUID PK), university_id (FK), seller_id (FK users), title, description, price, condition, category_id (FK), images (text[]), status (active/sold/removed).
2. **p2p_requests:** Buyer buy/contact requests. Columns: id (UUID PK), listing_id (FK), buyer_id (FK), message, status (pending/accepted/declined).
3. **p2p_orders:** P2P confirmed sales. Columns: id (UUID PK), university_id (FK), listing_id (FK), request_id (FK), buyer_id (FK), seller_id (FK), amount, status.
4. **rental_items:** Shared equipment catalog. Columns: id (UUID PK), university_id (FK), owner_id (FK users), title, description, category_id (FK), price_per_day, deposit, late_fee_policy (JSONB), available_from, available_to, images (text[]), status (active/rented/maintenance/removed).

5. **rental_bookings:** Equipment rental reservations. Columns: id (UUID PK), university_id (FK), rental_item_id (FK), borrower_id (FK users), start_date, end_date, status (pending_payment/confirmed/active/completed/cancelled/disputed), deposit_status (held/released/forfeited/partially_refunded), late_fee, total_amount, picked_up_at, returned_at, return_condition, owner_confirmed_at, renter_confirmed_at.

Domain D: Campus Events, Ticketing and Admissions (7 Tables)

1. **clubs:** Student clubs & campus organizers. Columns: id (UUID PK), university_id (FK), name, lead_user_id (FK users), status (active/suspended).
2. **events:** Campus event records. Columns: id (UUID PK), university_id (FK), club_id (FK clubs), title, description, category, date_time, end_time, venue_name, venue_address, location, capacity, poster_url, status (draft/pending_review/approved/published/completed/cancelled), reject_reason, recurrence_rule, parent_event_id (FK).
3. **ticket_types:** Event pricing & quota tiers. Columns: id (UUID PK), university_id (FK), event_id (FK events), name, price, quantity_total, quantity_sold, description, is_active, purchase_limit, sales_start_at, sales_end_at.
4. **tickets:** Issued digital event passes. Columns: id (UUID PK), university_id (FK), event_id (FK events), ticket_type_id (FK ticket_types), holder_id (FK users), ticket_number (text, e.g., 'GIB-F7A79799'), qr_token_hash (text), status (pending_payment/active/used/cancelled/refunded/void), used_at, used_by (FK users).
5. **ticket_checkins:** Entry gate admission audit log. Columns: id (UUID PK), ticket_id (FK tickets), event_id (FK events), staff_id (FK users), checked_in_at, device_info, is_reversal (bool), reversed_by (FK), reversed_at, notes.
6. **waitlist_entries:** Sold-out event ticket waitlists. Columns: id (UUID PK), university_id (FK), event_id (FK), ticket_type_id (FK), user_id (FK), status (waiting/notified/claimed/expired), notified_at.
7. **organizer_applications:** Club lead application queue. Columns: id (UUID PK), user_id (FK), university_id (FK), organizer_name, organization_name, organizer_type, phone, document_urls (text[]), status (submitted/approved/rejected), reviewed_by (FK).

Domain E: Payments, Commissions and Settlements (7 Tables)

1. **payments:** Abstract payment ledger. Columns: id (UUID PK), university_id (FK), transaction_type (order/ticket/booking/p2p_order/promotion), transaction_id (UUID), provider (manual/mock/chapa), amount, currency (ETB), status (pending/under_review/confirmed/failed/refunded), payer_id (FK users), payee_id (FK users), payee_accounts (JSONB), confirmed_at, expires_at, proof_attempts.
2. **payment_proofs:** Uploaded payment verification receipts. Columns: id (UUID PK), university_id (FK), payment_id (FK payments), payer_id (FK), payee_id (FK), file_name, image_sha256, reference_number, status (submitted/verified/rejected), reviewer_id (FK), reviewed_at, reject_reason.
3. **payout_accounts:** Bank/mobile money accounts. Columns: id (UUID PK), university_id (FK), owner_id (FK users), method (CBE/Telebirr/Awash/BOA), account_name, account_number, instructions, is_default (bool), active (bool), is_campus_hub (bool).
4. **commissions:** Platform revenue ledger. Columns: id (UUID PK), university_id (FK), payment_id (FK payments), transaction_type, transaction_id, payee_id (FK users), payee_role, gross_amount, base_amount, rate (numeric; for example 0.05, 0.025, 0.00), commission_amount, status (due/settled/void), settled_at, settled_by (FK users), settle_note.
5. **commission_payments:** Merchant-to-platform settlement submissions. Columns: id (UUID PK), university_id (FK), merchant_id (FK users), amount, verified_amount, reference_number, proof_file_name, status (submitted/verified/rejected), reviewer_id (FK), reviewed_at, notes.
6. **organizer_payouts:** Platform-to-organizer ticket revenue payouts. Columns: id (UUID PK), university_id (FK), organizer_id (FK users), event_id (FK events), amount (97.5% net), status (pending/completed), reference_number, recorded_by (FK users).
7. **transactions:** Records of payment captures, refunds, and reversals. Columns: id (UUID PK), payment_id (FK), type (capture/refund/reversal), amount.

Domain F: Governance, Feedback and Auditing (8 Tables)

1. **fee_schedules:** University-specific commission overrides. Columns: id (UUID PK), university_id (FK), commission_rate, featured_rate_per_day, effective_from, created_by (FK).
2. **agreement_versions:** Terms of service versions. Columns: id (UUID PK), agreement_type (merchant_terms/organizer_terms), version, title, intro, content (JSONB), effective_from, is_active, created_by (FK).
3. **application_agreements:** User agreements on application submission. Columns: id (UUID PK), user_id (FK), university_id (FK), application_type, application_id, agreement_type, agreement_version, commission_rate, agreed_at.
4. **notifications:** In-app actionable notifications. Columns: id (UUID PK), user_id (FK users), type, title, body, read_at, target_route, target_entity_type, target_entity_id, created_at.
5. **reviews:** User feedback with polymorphic targets. Columns: id (UUID PK), university_id (FK), author_id (FK users), target_type (store/product/listing/user), target_id (UUID), rating (1-5 int), comment, created_at.
6. **reports:** Moderation flagging. Columns: id (UUID PK), university_id (FK), reporter_id (FK users), target_type (product/listing/user/store/event), target_id (UUID), reason, details, status (open/resolved/dismissed), resolution.
7. **disputes:** Rental and transaction dispute queue. Columns: id (UUID PK), university_id (FK), booking_id (FK), raised_by (FK), reason, evidence (text[]), status (open/under_review/resolved), resolution.
8. **audit_logs:** System-wide audit log; append-only at the application level (NFR-014). Columns: id (UUID PK), university_id (FK), actor_id (FK users), action, target_type, target_id, details (JSONB), created_at.

8. System Process Flows and Data Flow Diagrams

8.1 Level 0: Context Data Flow Diagram

Figure 4 shows Gibiyachin as a single system with its external entities: students, campus merchants, event organizers, and administrators (Campus Administrators and Super Administrators are shown together). Bank and mobile-money transfers are made outside the system, and users upload proof of payment. The email service delivers one-time codes and reset links.

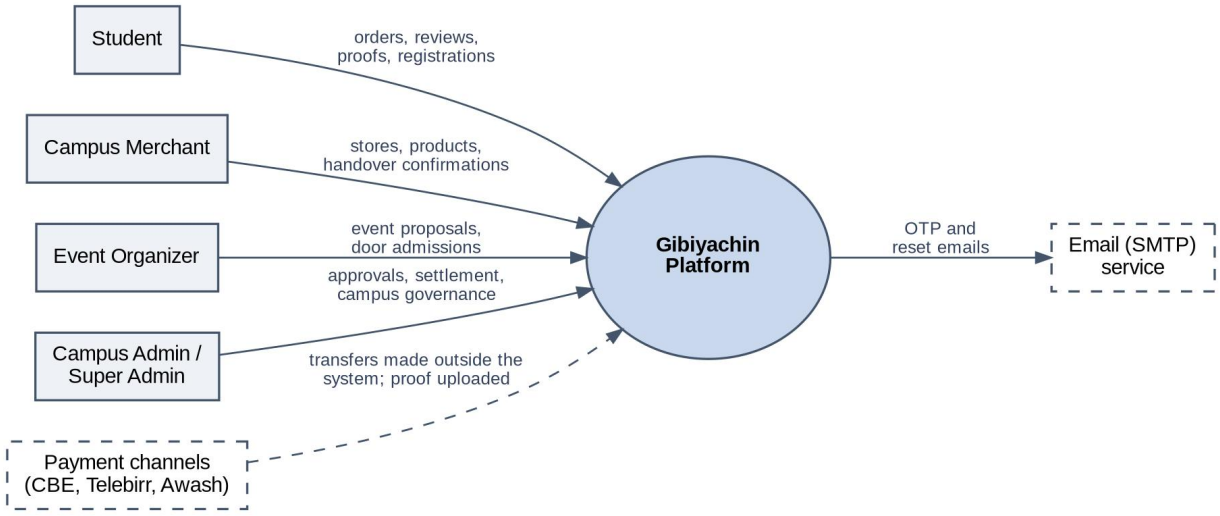


Figure 4: Level 0 context data flow diagram

8.2 Level 1: Subsystem Data Flow Diagram

Figure 5 divides the system into six main processes and the data stores they use. It covers authentication, orders, payments, ticketing, check-in, and governance. The P2P, rental, verification, and notification features are not broken down in this diagram.

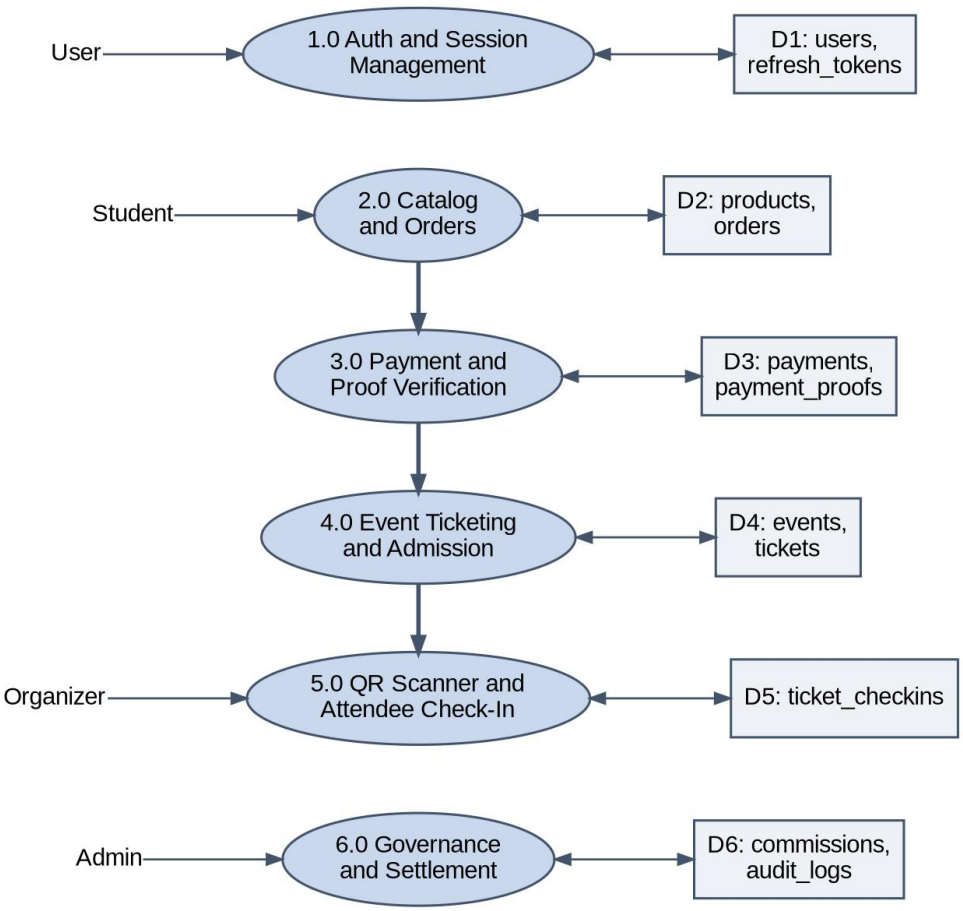


Figure 5: Level 1 data flow diagram

8.3 Level 2: Core Transaction Process Flows

A. Material Order Fulfillment Flow (5.0% Commission)

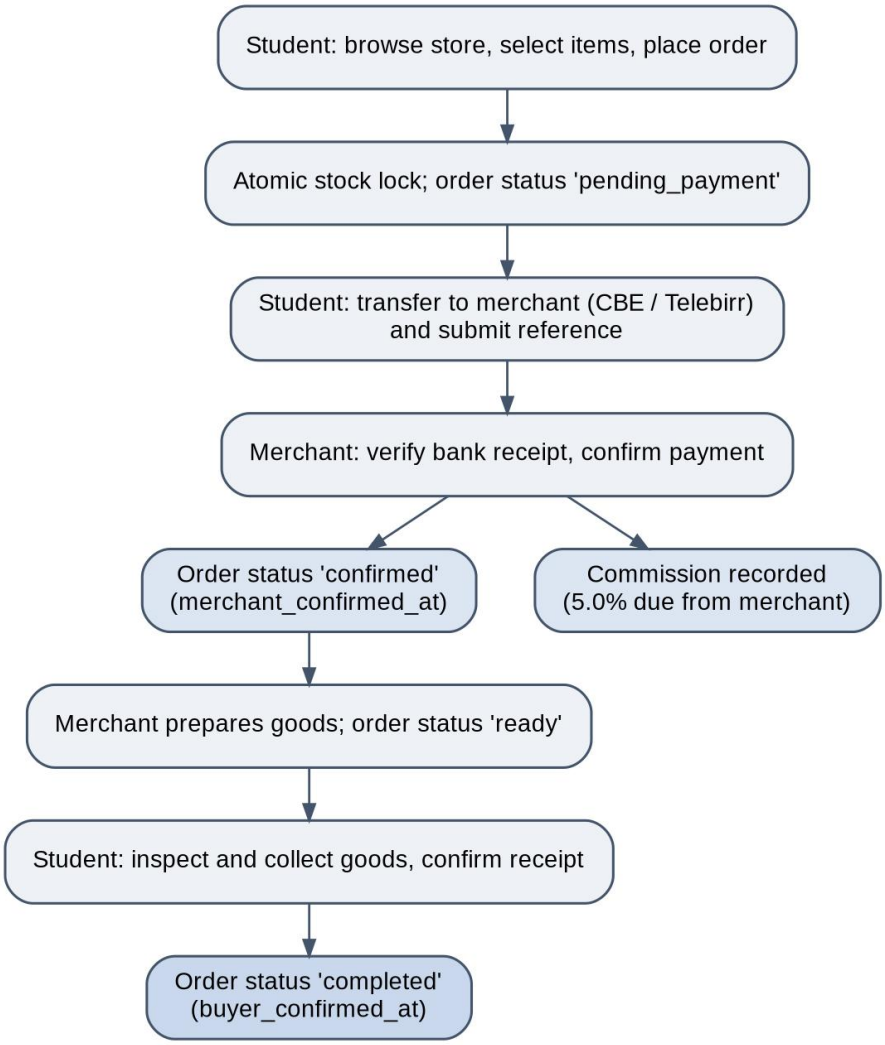


Figure 6: Material order fulfillment flow

B. Event Ticket Admission Flow (2.5% Commission)

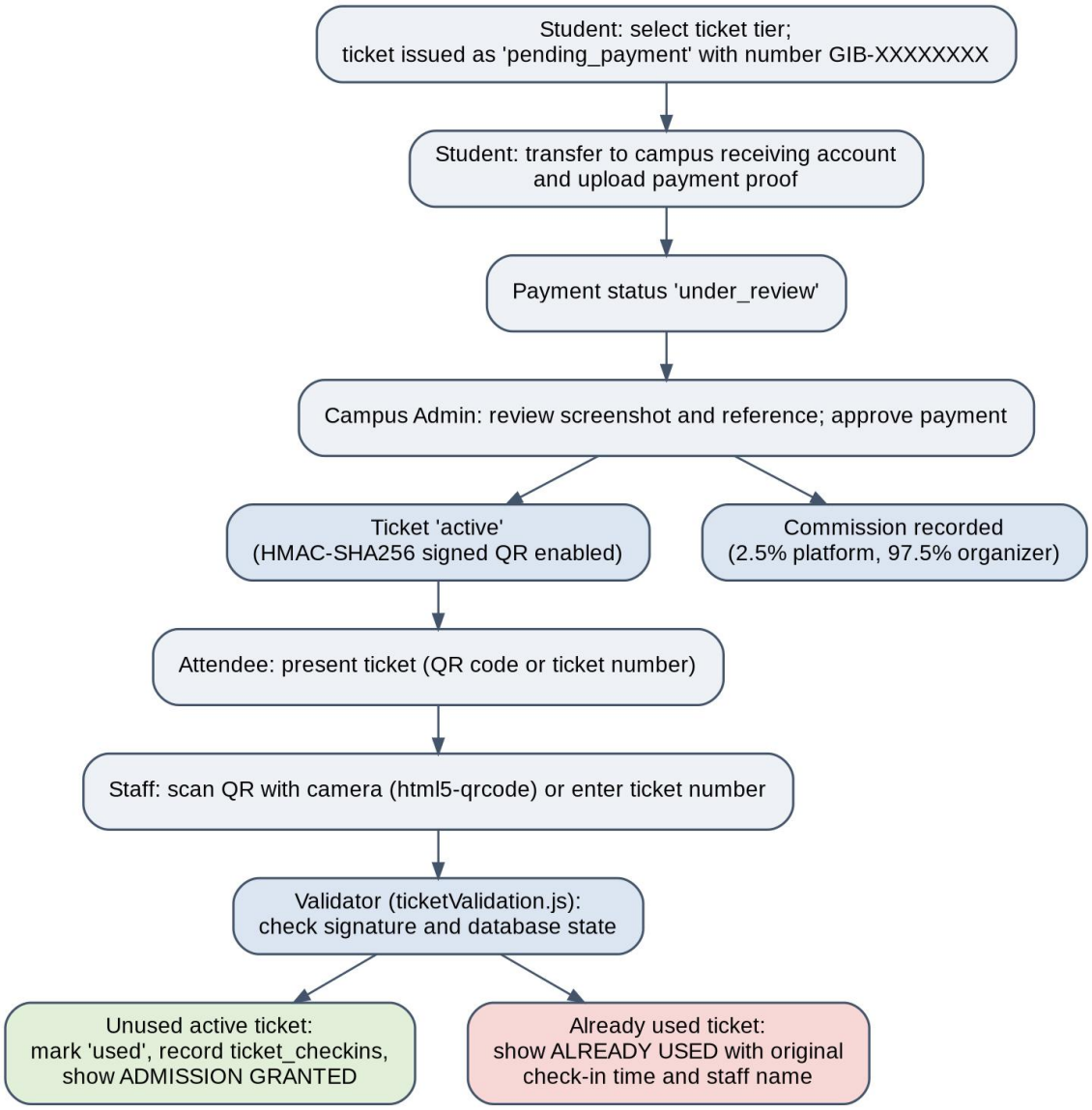


Figure 7: Event ticket purchase and admission flow

9. Core Use Cases

This section describes 15 use cases for the main workflows. Each one lists the requirements it relates to. Requirements without a use case are shown in Table 6.

UC-01: User Registration and Email OTP Verification

- **Actor:** Prospective Student or Merchant.
- **Preconditions:** User has a valid university or personal email.
- **Trigger:** User completes registration form and clicks “Sign Up”.
- **Main Scenario:**

1. System checks domain against universities.domain and permitted_domains.
2. Creates account with email_verified = false.
3. Dispatches 6-digit OTP code to user’s email.
4. User receives code, inputs it into verification screen.
5. System verifies SHA-256 hash and TTL.
6. Unlocks account (email_verified = true).
   - **Postconditions:** User may log in to platform.

Related requirements: FR-001, FR-002.

UC-02: User Login and Token Refresh Rotation

- **Actor:** Registered User.
- **Trigger:** Submits email and password.
- **Main Scenario:**

1. System verifies Bcrypt hash and checks suspended == false.
2. Issues 15-minute JWT access token.
3. Sets the HttpOnly refresh token cookie and the CSRF cookie (the client sends the CSRF value back in the X-CSRF-Token header).
4. Frontend sends periodic refresh requests to maintain session.
   - **Postconditions:** Active session maintained securely.

Related requirements: FR-003, FR-004, FR-006.

UC-03: Profile Customization and Privacy Shielding

- **Actor:** Authenticated User.
- **Main Scenario:**

1. User edits bio, department, year, and uploads avatar/cover photos.
2. Uploaded images are re-encoded via Sharp, stripping EXIF tags.
3. User toggles showEmail: false and showPhone: false.

4. When a visitor or a user from another campus views the public profile, department and year are hidden, and email and phone appear only if the owner has allowed them.
   - **Postconditions:** Profile updated; the privacy settings apply to public views.

Related requirements: FR-007, FR-011.

UC-04: Merchant Product Creation and Stock Tracking

- **Actor:** Approved Merchant.
- **Main Scenario:**

1. Merchant inputs product details, price in Birr, stock quantity, and images.
2. Product listed in university marketplace.
3. When stock is purchased, quantity decrements automatically.
4. When stock reaches 0, system marks product ‘out_of_stock’.
   - **Postconditions:** Product catalog accurately reflects real-time inventory.

Related requirements: FR-014.

UC-05: Order Placement and Physical Handover Handshake

- **Actor:** Student Buyer and Campus Merchant.
- **Main Scenario:**

1. Student places order, atomically locking stock.
2. Student pays merchant directly via CBE/Telebirr.
3. Merchant marks payment received (merchant_confirmed_at).
4. Merchant marks order ready for pickup.
5. Student collects physical material and clicks “Confirm Receipt” (buyer_confirmed_at).
6. Order finalized as completed (the 5.0% commission was already recorded when the payment was confirmed).
   - **Postconditions:** Both parties confirm exchange; ledger updated.

Related requirements: FR-017, FR-018.

UC-06: Order Cancellation and Stock Restocking

- **Actor:** Buyer or Merchant.
- **Main Scenario:**

1. Buyer requests cancellation of unfulfilled order.
2. If the order was already paid, the Merchant or an Admin approves the cancellation; an unpaid order can be cancelled by the buyer immediately.
3. System releases stock back to product catalog.
   - **Postconditions:** Order marked cancelled; inventory restored.

Related requirements: FR-019.

UC-07: P2P Classified Listing and Negotiation

- **Actor:** Student Seller and Buyer.
- **Main Scenario:**

1. Student posts used textbook with photos and condition rating.
2. Buyer submits buy request with meeting location message.
3. Seller accepts the request. The buyer pays the accepted order in the app (FR-050), and the seller hands over the item.
    - **Postconditions:** The listing is marked sold after payment is confirmed.

Related requirements: FR-021, FR-022, FR-050.

UC-08: Equipment Rental Booking and Return Lifecycle

- **Actor:** Borrower and Equipment Owner.
- **Main Scenario:**

1. Borrower books calculator for 3 days; deposit calculated.
2. Owner confirms direct payment.
3. Borrower confirms pickup (picked_up_at = now()).
4. Borrower returns item on time; owner confirms return condition.
5. Deposit released back to borrower.
    - **Postconditions:** Rental completed with 0% platform fee.

Related requirements: FR-023, FR-024, FR-025, FR-026.

UC-09: Event Proposal and Campus Admin Review

- **Actor:** Club Organizer and Campus Admin.
- **Main Scenario:**

1. Organizer submits event details, date, venue, and poster.
2. Campus Admin reviews safety and institutional policy.
3. Admin approves event, publishing ticket sales to students.
    - **Postconditions:** Event visible on campus events feed.

Related requirements: FR-028, FR-029.

UC-10: Ticket Purchase and Manual Payment Verification

- **Actor:** Student Buyer and Campus Admin.
- **Main Scenario:**

1. Student selects ticket tier; ticket created in pending_payment.
2. Student transfers ticket price to university account.
3. Student uploads screenshot receipt and reference number.
4. Campus Admin reviews proof and approves payment.
5. Ticket status becomes active; HMAC-SHA256 signed QR code generated.

6. 2.5% platform commission recorded.
   - **Postconditions:** Attendee holds verified digital ticket.

Related requirements: FR-031, FR-032, FR-033, FR-038.

UC-11: Gate Admission Scan and Repeat Entry Denial

- **Actor:** Event Staff and Attendee.
- **Main Scenario:**

1. Attendee presents signed QR or ticket number (GIB-XXXXXXXX).
2. Staff scans QR via device camera.
3. Central service checks HMAC validity and database state.
4. If unused: marks ticket used, records ticket_checkins, admits attendee.
5. If already used: alerts staff with red “ALREADY USED” banner.
   - **Postconditions:** One-time admission strictly enforced.

Related requirements: FR-034, FR-035.

UC-12: Admission Reversal

- **Actor:** Event Organizer or Campus Admin.
- **Main Scenario:**

1. Staff accidentally scans attendee who steps out of line.
2. Staff clicks “Reverse Check-In” on attendee roster.
3. Ticket status is reset to active, and a reversal record (with optional notes) is added to ticket_checkins.
   - **Postconditions:** Attendee eligible for re-entry.

Related requirements: FR-036.

UC-13: Merchant Commission Settlement

- **Actor:** Merchant and Campus Admin.
- **Main Scenario:**

1. Merchant views accumulated 5.0% commission balance.
2. Merchant pays Gibiyachin receiving account and submits proof.
3. Admin verifies proof and triggers settlement.
4. System allocates payment chronologically to oldest due entries.
   - **Postconditions:** Merchant commission balance cleared.

Related requirements: FR-038, FR-039.

UC-14: Campus Verification Document Review

- **Actor:** Student and Campus Admin.
- **Main Scenario:**

1. Student with Gmail account uploads student ID card image.
2. Admin reviews ID photo in verification queue.
3. Admin approves request; user upgraded to verified status.
    - **Postconditions:** Student gains full platform access.

Related requirements: FR-041.

UC-15: Super Admin University Provisioning

- **Actor:** Super Administrator.
- **Main Scenario:**

1. Super Admin creates university record (AASTU) with domain, city, and code.
2. Assigns initial Campus Administrator account.
3. System provisions institutional tenant.
    - **Postconditions:** University tenant ready for campus community.

Related requirements: FR-042.

10. Security Architecture and Cryptographic Controls

This section summarizes the security controls described elsewhere in this document. It says what the system is designed to do and what the tests cover. It is not a security audit, and no formal audit or certification is reported here. Section 14.3 lists known weaknesses. The controls are described with reference to common web-application risks such as those in the OWASP Top 10 (2021) [5].

10.1 Authentication and Session Security

- **Bcrypt Password Hashing:** Salted hashes generated with cost factor 10, which makes precomputed (rainbow-table) attacks ineffective and slows down password guessing.
- **Short-Lived Access Tokens:** Signed with HMAC-SHA256 (jsonwebtoken), expiring in 15 minutes to limit exposure if intercepted.
- **HttpOnly Refresh Cookies:** Refresh tokens stored in cookies inaccessible to JavaScript (HttpOnly: true, SameSite: Lax, Path: /api/auth), so that scripts cannot read them, which reduces the risk of token theft through Cross-Site Scripting (XSS).
- **Double-Submit CSRF Defense:** Cookie-authenticated endpoints (refresh and logout) validate matching CSRF tokens using crypto.timingSafeEqual to avoid timing side channels. Bearer-authenticated mutation endpoints are not cookie-driven and therefore do not require a CSRF token.
- **Family-Based Refresh Token Theft Detection:** On token rotation, the old token is marked revoked. If an attacker replays a revoked token, the backend revokes the entire family_id immediately.

10.2 Cryptographic QR Code Verification and Key Rotation

- **HMAC-SHA256 Payload:** Ticket QR codes (HMAC is defined in RFC 2104 [4]) encode <ticket_id>.<signature> generated using server-side secret TICKET_QR_SECRET.
- **Timing-Safe Comparison:** Signatures are verified using crypto.timingSafeEqual on byte buffers of equal length.
- **Zero-Downtime Key Rotation:** Validator checks TICKET_QR_SECRET, falling back to TICKET_QR_SECRET_PREVIOUS if present.
- **Admission vs. Signature Separation:** A valid HMAC signature confirms authenticity, but admission is granted only if the ticket's database status is 'active' with used_at IS NULL.

10.3 Insecure Direct Object Reference (IDOR) and Tenant Isolation

- **Tenant Scoping:** All database queries append university_id = $X derived from the verified user session (req.user.university_id), preventing cross-campus data access.
- **Entity Authorization:** Accessing orders, bookings, and payment proofs verifies that req.user.id matches either the buyer or seller. Unauthorized access attempts return a generic 404 Not Found so that the response does not reveal whether the record exists.

10.4 Payment Verification and Rate Limiting

- **Payment verification:** Manual payments are confirmed only by the receiving party or an administrator (FR-018, FR-032, FR-039). Proof images are stored privately (Section 11). The `payment_proofs` table stores an image hash (`image_sha256`) and a reference number, and the recorded test run includes a duplicate-reference check (Section 15.1).
- **Rate limiting:** Login is limited to 8 requests per minute per IP address and email (FR-003). Email OTP resends have a 60-second cooldown and a limit of 5 per hour (FR-002). Batch check-in is limited to 20 requests per minute per user (FR-047). Without Redis, the counters are kept per process (Section 14.3).

11. File Upload and Storage Management

Uploaded files are validated and processed to reduce the risk of malicious uploads, such as files that are disguised as images and oversized images.

11.1 Processing Pipeline

Client upload → Multer temporary staging → magic-byte validation → Sharp decode and sanitize → random file name → storage

1. **Multer Temp Staging:** Files stream to uploads_temp/ with random filenames (tmp_<hex>). In-memory storage is avoided to prevent memory exhaustion on large PDFs.
2. **Magic-Byte Sniffing:** The first file bytes are inspected against known signatures:
   - JPEG: 0xFF, 0xD8, 0xFF
   - PNG: 0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A, 0x1A, 0x0A
   - WebP: RIFF....WEBP
   - GIF: GIF87a / GIF89a
   - PDF: %PDF (0x25, 0x50, 0x44, 0x46)
3. **Image Sanitization with Sharp:**
   - Decodes pixel data, discarding client-supplied headers and stripping EXIF metadata.
   - Converts animated GIFs to static PNGs.
   - Limits dimensions to 8192 x 8192 pixels to reduce the risk of decompression bombs.
   - Re-encodes JPEGs at quality 90 and PNGs at compression level 6.
4. **Randomized Renaming:** Client filenames are discarded. Destination filenames use 20-byte random hex strings (crypto.randomBytes(20).toString('hex') + ext).
5. **Private vs. Public Storage Separation:**
   - Public assets (avatars, product photos, event posters) reside in /uploads/ served with X-Content-Type-Options: nosniff.
   - Sensitive documents (student IDs, payment proof screenshots) reside in /private_uploads/proofs/, accessible only via authenticated endpoints with Cache-Control: private, no-store.

12. Business Model Canvas and Revenue Streams

This section outlines the business model planned for the pilot. It is a planning description. No revenue figures, user numbers, or partnership agreements are reported in this document. Table 5 follows the Business Model Canvas structure.

Table 5: Business Model Canvas for Gibiyachin

<table>
<tr>
<td rowspan="2">Key Partners (planned or assumed)
Ethiopian public universities (AASTU, AAU)
Campus merchants
Student unions and clubs
Commercial Bank of Ethiopia (CBE)
Ethio Telecom (Telebirr)</td>
<td rowspan="2">Key Activities and Key Resources
Activities: marketplace operations, order fulfillment, ticket validation and gate admission, ID verification, dispute handling
Resources: Express/PostgreSQL application, QR validation service, university data</td>
<td rowspan="2">Value Propositions
Verified campus community
QR-based event entry that is hard to forge
Clear commission rules (5% / 2.5% / 0%)
Interface designed for mobile and slow connections</td>
<td>Customer Relationships and Channels
Relationships: self-service marketplace, in-app notifications, Campus Admin moderation
Channels: web application, campus communities, student Telegram groups</td>
<td rowspan="2">Customer Segments
University students
Campus merchants
Student club organizers
University administrative staff</td>
</tr>
<tr>
<td>Revenue Streams
5.0% commission on merchant material orders
2.5% commission on event ticket sales (97.5% to the club)
0.0% commission on equipment rentals
Featured-product promotion fees</td>
</tr>
<tr>
<td colspan="3">Cost Structure
Cloud VPS and PostgreSQL hosting (ETB)
Storage for media and payment proofs
Domain registration and SSL certificates
Administrative review effort</td>
<td colspan="2"></td>
</tr>
</table>

12.1 Revenue Streams Breakdown

1. **Merchant Material Orders (5.0% Commission):** Recorded when the merchant confirms payment on an order and billed to the merchant. The buyer pays the merchant directly, and the merchant settles the accumulated commission with the platform periodically.
2. **Event Ticket Sales (2.5% Commission):** Deducted from ticket revenue. For a 100 ETB ticket, the platform retains 2.50 ETB, and the student club/organizer receives 97.50 ETB via manual payout.
3. **Equipment Rentals (0.0% Commission):** Provided as a free resource-sharing service to encourage student use.
4. **In-App P2P Orders:** These use the fallback commission rate of the university fee schedule (default 5.0%). The campus decision on a dedicated P2P rate is still pending (FR-038, Section 14.3).
5. **Featured Product Promotion:** Merchants pay a daily fee (default 20 ETB per day when no fee schedule exists). No seller commission is recorded for promotion payments (FR-046).

13. Requirements Traceability Matrix (RTM)

Table 6 links each functional requirement to its implementation file, database tables, related use case, and test evidence. File paths and test names are taken from the project repository records. The last column separates automated tests that were part of the recorded test run (Section 15.1) from shell scripts and manual checks, which were not part of that run. A use case of "None" means that no use case in Section 9 covers the requirement.

Table 6: Requirements traceability matrix

ID	Module	Implementation source	Database tables	Use case	Test evidence	Evidence type
FR-001	Auth	`backend/src/modules/auth/routes.js`	users, universities, email_otps	UC-01	tenant_auth.test.js	Automated test
FR-002	Auth	`backend/src/services/otp.js`	email_otps, users	UC-01	security_phase1.test.js	Automated test
FR-003	Auth	`backend/src/modules/auth/routes.js`	users, refresh_tokens	UC-02	security_phase1.test.js	Automated test
FR-004	Auth	`backend/src/modules/auth/routes.js`	refresh_tokens, users	UC-02	security_phase1.test.js	Automated test
FR-005	Auth	`backend/src/modules/auth/routes.js`	users, refresh_tokens	None	forgot_password_reset.test.js	Automated test
FR-006	Auth	`backend/src/modules/auth/routes.js`	refresh_tokens	UC-02	security_phase1.test.js	Automated test
FR-007	Profile	`backend/src/modules/auth/users.routes.js`	users, universities	UC-03	admin_user_management.test.js	Automated test
FR-008	Profile	`backend/src/modules/auth/users.routes.js`	users, refresh_tokens	None	security_phase1.test.js	Automated test
FR-009	Profile	`backend/src/modules/auth/users.routes.js`	orders, tickets, stores	None	Manual / Route Scoping	Manual check
FR-010	Profile	`backend/src/modules/auth/users.routes.js`	orders, tickets, listings	None	Manual Verification	Manual check
FR-011	Profile	`backend/src/modules/auth/users.routes.js`	users, stores, events	UC-03	tenant_route_scoping.test.js	Automated test
FR-012	Onboard	`backend/src/modules/merchants/routes.js`	merchant_applications, application_agreements, agreement_versions	None	onboarding_applications.test.js	Automated test
FR-013	Store	`backend/src/modules/merchants/routes.js`	stores	None	tenant_route_scoping.test.js	Automated test
FR-014	Product	`backend/src/modules/products/routes.js`	products	UC-04	tenant_route_scoping.test.js	Automated test
FR-015	Product	`backend/src/modules/products/routes.js`	products, stores	None	tenant_route_scoping.test.js	Automated test
FR-016	Product	`backend/src/modules/products/routes.js`	favorites	None	Manual UI Verification	Manual check
FR-017	Orders	`backend/src/modules/orders/routes.js`	orders, order_items, products	UC-05	smoke.sh	Shell script (not in recorded run)

ID	Module	Implementation source	Database tables	Use case	Test evidence	Evidence type
FR-018	Orders	`backend/src/modules/orders/routes.js`	orders, order_status_history	UC-05	manual_payments_system.test.js	Automated test
FR-019	Orders	`backend/src/modules/orders/routes.js`	orders, products	UC-06	manual_payments_system.test.js	Automated test
FR-020	Orders	`backend/src/modules/orders/routes.js`	orders, order_status_history	None	manual_payments_system.test.js	Automated test
FR-021	P2P	`backend/src/modules/listings/routes.js`	p2p_listings	UC-07	smoke.sh	Shell script (not in recorded run)
FR-022	P2P	`backend/src/modules/listings/routes.js`	p2p_requests	UC-07	smoke.sh	Shell script (not in recorded run)
FR-023	Rentals	`backend/src/modules/rentals/routes.js`	rental_items	UC-08	smoke_phase2.sh	Shell script (not in recorded run)
FR-024	Rentals	`backend/src/modules/rentals/routes.js`	rental_bookings	UC-08	smoke_phase2.sh	Shell script (not in recorded run)
FR-025	Rentals	`backend/src/modules/rentals/routes.js`	rental_bookings	UC-08	smoke_phase2.sh	Shell script (not in recorded run)
FR-026	Rentals	`backend/src/modules/rentals/routes.js`	disputes, rental_bookings	UC-08	smoke_phase2.sh	Shell script (not in recorded run)
FR-027	Onboard	`backend/src/modules/organizers/routes.js`	organizer_applications, clubs	None	onboarding_applications.test.js	Automated test
FR-028	Events	`backend/src/modules/events/routes.js`	events, clubs	UC-09	event_ticketing_system.test.js	Automated test
FR-029	Events	`backend/src/modules/events/routes.js`	events	UC-09	event_ticketing_system.test.js	Automated test
FR-030	Tickets	`backend/src/modules/events/routes.js`	ticket_types	None	event_ticketing_system.test.js	Automated test
FR-031	Tickets	`backend/src/modules/tickets/purchase.routes.js`	tickets, payments	UC-10	event_ticketing_system.test.js	Automated test
FR-032	Tickets	`backend/src/modules/payments/manual.routes.js`	tickets, payments, commissions	UC-10	event_ticketing_system.test.js	Automated test
FR-033	Tickets	`backend/src/utils/qr.js`	tickets	UC-10	event_ticketing_system.test.js	Automated test
FR-034	Tickets	`backend/src/services/ticketValidation.js`	tickets, events	UC-11	event_ticketing_system.test.js	Automated test
FR-035	Tickets	`backend/src/services/ticketValidation.js`	tickets, ticket_checkins	UC-11	event_ticketing_system.test.js	Automated test
FR-036	Tickets	`backend/src/services/ticketValidation.js`	tickets, ticket_checkins	UC-12	event_ticketing_system.test.js	Automated test
FR-037	Payments	`backend/src/modules/payments/payout.routes.js`	payout_accounts	None	manual_payments_system.test.js	Automated test
FR-038	Ledger	`backend/src/services/commissions.js`	commissions	UC-10, UC-13	manual_payments_system.test.js	Automated test

ID	Module	Implementation source	Database tables	Use case	Test evidence	Evidence type
FR-039	Settle	`backend/src/modules/commissions/routes.js`	commission_payments, commissions	UC-13	manual_payments_system.test.js	Automated test
FR-040	Notify	`backend/src/utils/notify.js`	notifications	None	actionable_notifications.test.js	Automated test
FR-041	Verify	`backend/src/modules/verifications/routes.js`	verification_requests	UC-14	tenant_route_scoping.test.js	Automated test
FR-042	Super	`backend/src/modules/super/routes.js`	universities, users	UC-15	tenant_integrity.test.js	Automated test
FR-043	Tickets	`backend/src/modules/tickets/purchase.routes.js`	waitlist_entries	None	smoke_phase2.sh	Shell script (not in recorded run)
FR-044	Reviews	`backend/src/modules/reviews/routes.js`	reviews, orders, rental_bookings	None	reviews_and_p2p_payment.test.js	Automated test
FR-045	Moderation	`backend/src/modules/reports/routes.js`	reports, audit_logs	None	tenant_route_scoping.test.js	Automated test
FR-046	Product	`backend/src/modules/products/routes.js`	products, payments	None	tenancy.sh	Shell script (not in recorded run)
FR-047	Tickets	`backend/src/modules/tickets/routes.js`	tickets, ticket_checkins	None	smoke_phase2.sh	Shell script (not in recorded run)
FR-048	Admin	`backend/src/modules/admin/routes.js`	orders, transactions, users	None	smoke_phase2.sh	Shell script (not in recorded run)
FR-049	Events	`backend/src/modules/events/routes.js`	events	None	smoke_phase2.sh	Shell script (not in recorded run)
FR-050	P2P	`backend/src/modules/listings/routes.js`	p2p_orders, p2p_listings, payments	UC-07	reviews_and_p2p_payment.test.js	Automated test
FR-051	Super	`backend/src/modules/super/routes.js`	fee_schedules, agreement_versions	None	tenancy.sh	Shell script (not in recorded run)
FR-052	Admin	`backend/src/modules/admin/routes.js`	users, audit_logs	None	admin_user_management.test.js	Automated test

14. Implementation Status and Future Roadmap

This section separates implemented features from planned ones and shows what evidence exists for each implemented feature. Implementation status is taken from the project repository records. This document does not report production deployment, live-payment verification, or user-load results.

14.1 Implemented Features and Evidence

Table 7 lists each feature area with the evidence that exists for it. “Automated tests” means the automated backend tests listed in Table 8; “shell script or manual check” means evidence that was not part of that recorded run. Feature areas with weaker evidence are implemented but have not been fully tested.

Table 7: Implementation status and test evidence by feature area

Feature area	Requirements	Status and evidence
Registration, email OTP, login, session refresh, password reset, logout	FR-001 to FR-006	Automated tests (all passed in recorded run)
Profile view and update, password change, public profile with privacy masking	FR-007, FR-008, FR-011	Automated tests (all passed in recorded run)
Profile statistics and activity feed	FR-009, FR-010	Implemented; manual checks only
Merchant application, store, product catalog, marketplace browsing	FR-012 to FR-015	Automated tests (all passed in recorded run)
Favorite items	FR-016	Implemented; manual UI check only
Order placement with atomic stock hold	FR-017	Implemented; shell script only (not in recorded run)
Order payment confirmation, handover, cancellation, history	FR-018 to FR-020	Automated tests (all passed in recorded run)
P2P listings and buy requests	FR-021, FR-022	Implemented; shell script only (not in recorded run)
In-app payment for P2P orders	FR-050	Automated tests (all passed in recorded run)
Equipment rentals, deposits, late fees, disputes	FR-023 to FR-026	Implemented; shell script only (not in recorded run)
Club onboarding, event creation, administrative approval	FR-027 to FR-029	Automated tests (all passed in recorded run)
Ticket tiers, purchase, payment approval, QR generation, validation, check-in, reversal	FR-030 to FR-036	Automated tests (all passed in recorded run)
Manual payment accounts, commission rules, commission settlement	FR-037 to FR-039	Automated tests (all passed in recorded run)
Notifications with deep links; campus ID verification	FR-040, FR-041	Automated tests (all passed in recorded run)
Multi-campus tenant management and tenant isolation	FR-042, NFR-009	Automated tests (all passed in recorded run)
Ticket waitlist	FR-043	Implemented; shell script only (not in recorded run)

Feature area	Requirements	Status and evidence
Reviews and ratings; content reporting and moderation	FR-044, FR-045	Automated tests (all passed in recorded run)
Featured product promotion	FR-046	Implemented; shell script only (not in recorded run)
Offline scan queue and batch check-in	FR-047	Implemented; shell script only (not in recorded run)
CSV export	FR-048	Implemented; shell script only (not in recorded run)
Recurring event occurrences	FR-049	Implemented; shell script only (not in recorded run)
Fee schedules and agreement versioning	FR-051	Implemented; shell script only (not in recorded run)
Account deactivation and deletion	FR-052	Automated tests (all passed in recorded run)
File upload checks and image re-encoding	NFR-011, Section 11	Automated tests (all passed in recorded run)
Chapa payment provider and webhook	Section 2.7	Implemented but disabled by default; not validated against the live gateway
SMS delivery	Section 2.7	Simulated only; no gateway connected
Frontend user interface (all features)	NFR-015, NFR-016	Lint and production build succeed; no browser end-to-end tests; checked manually

14.2 Future Enhancements (Deferred to Subsequent Phases)

- **SMS Gateway:** Integration with an active Ethio Telecom SMS API gateway (currently simulated via notify.sendSms).
- **Automated Payment Gateway Webhooks:** Integration with Chapa or Telebirr API webhooks for automated payment reconciliation (architecture implemented in webhook.routes.js, awaiting production merchant gateway accounts).
- **Native Mobile Applications:** Standalone iOS and Android native apps built via React Native (the responsive web application currently serves mobile clients; Progressive Web App features such as a web manifest and service worker are not yet implemented).
- **In-App Direct Chat:** Real-time WebSocket messaging between buyers and sellers (currently handled via contact requests, public contact info, and phone links).
- **Escrow Digital Wallet:** In-app digital wallet for automated deposit holding and dispute chargebacks.

14.3 Known Limitations and Hardening Backlog

The limitations below are recorded so that this document does not overstate the system's readiness. They require review and follow-up before wider production use. A supervised demonstration is not the same as formal pilot acceptance; the criteria in Section 15.3 should be met before the pilot is described as accepted.

- **Development secret fallback:** the access-token module falls back to a built-in development value when JWT_ACCESS_SECRET is not set. The ticket QR module already refuses weak secrets in production; an equivalent start-up guard (minimum 32 characters) is required for JWT.
- **Content-Security-Policy:** the Helmet content security policy is disabled; it SHOULD be enabled for the frontend host before public launch.
- **Audit immutability:** append-only behaviour of audit_logs and ticket_checkins is enforced by application code only (NFR-014); database privileges or triggers SHOULD be added.
- **Rate limiting without Redis:** when REDIS_URL is unset, rate limits are counted per process and are therefore not shared between instances (NFR-020).
- **Rate-limit test coverage:** only the forgot-password limit has an automated test; the login and OTP limits and the Redis mode are untested.
- **Offline scan queue:** queued scans are kept in browser memory, so they are lost if the page is reloaded before synchronisation (FR-047).
- **Payment gateway and SMS:** the Chapa provider is implemented but untested against the live API, and SMS is simulated (Section 2.7).
- **Verification gaps:** performance NFR-002, NFR-012 and NFR-017 have not been measured, there are no browser end-to-end tests, and the shell smoke scripts are not run automatically (Section 15.2).
- **Commission policy for P2P sales:** in-app P2P orders currently use the fallback fee-schedule rate (FR-038). The campus business decision on a dedicated P2P rate is pending.

15. Acceptance Criteria and System Verification Summary

15.1 Test Execution Summary

The project team recorded the following results on 21 October 2026 against repository commit 9ab2b65. The tests ran on a clean database built from schema.sql and migrations 003 to 021 (Ubuntu 24.04, Node.js v22.22.2, PostgreSQL 16.13). All 323 automated backend tests passed (Table 8). The results are reproduced here as recorded.

Table 8: Automated backend test results from the recorded run

Area	Test files (.test.js)	Passed	Scope
Event ticketing	`event_ticketing_system`	41 / 41	HMAC validation, ticket-number parsing, one-time admission, repeat-scan rejection, reversal, roster
Multi-tenant isolation	`tenant_integrity`, `tenant_auth`, `tenant_route_scoping`, `migration_013`	84 / 84 (11 + 24 + 45 + 4)	No cross-tenant leakage, header scoping, tenant immutability
Security	`security_phase1`, `security_phase2`	60 / 60 (33 + 27)	CSRF, timing-safe comparison, SQL injection, IDOR, rate limiting
Manual payments and commissions	`manual_payments_system`	47 / 47	Proof upload, image re-encoding, duplicate reference protection, ticket approval, 5.0% / 2.5% / 0% rules
Actionable notifications	`actionable_notifications`	8 / 8	Allowlisted deep links, open-redirect prevention, read receipts
Password reset	`forgot_password_reset`	11 / 11	Hashed one-hour tokens, session revocation, enumeration resistance
Onboarding applications	`onboarding_applications`	26 / 26	Merchant and organizer applications, agreements, review decisions
Admin user management	`admin_user_management`	28 / 28	Profile, deactivation, dependency checks, deletion
Reviews and P2P payment	`reviews_and_p2p_payment`	18 / 18	Review validation and eligibility rules, accepted-request order, payout-account guard, payment record
Total		**323 / 323**	

Frontend quality: oxlint reports 0 errors and 70 advisory warnings, and the Vite build succeeds. The initial JavaScript bundle is 299.84 kB gzip and the stylesheet is 13.87 kB gzip (313.7 kB combined), which is within NFR-004.

Not part of this run: the shell smoke scripts (smoke.sh, smoke_phase2.sh, tenancy.sh, tenancy_phase2.sh) and manual UI checks. Several requirements are covered only by those scripts or by manual checks (see Table 6 and Table 7). There are no browser end-to-end tests.

15.2 Non-Functional Requirement Verification Status

Table 9 lists each non-functional requirement with the evidence that exists today. A status of "Not yet measured" or "Operational target" means the value in Section 6 is a target without a measured result. Targets still need to be confirmed in the pilot load test.

Table 9: Verification status of the non-functional requirements

ID	Evidence	Status
NFR-001	GET /api/products benchmark: 100 concurrent connections for 10 s on a development machine with seeded data; median 179 ms, P97.5 339 ms, P99 384 ms, about 515 requests/s. P95 was not reported separately.	Partial: 100 connections tested against a 200-user target; pilot load test pending.
NFR-002	Mutations run in short row-locked transactions; functional behaviour covered by tests; latency not measured.	Not yet measured
NFR-003	86 index definitions across schema and migrations; query plans not reviewed.	Partially verified
NFR-004	Production build output 313.7 kB gzip.	Met
NFR-005	Helmet default HSTS header; TLS is terminated by the reverse proxy at deployment.	Deployment configuration
NFR-006	bcrypt cost factor 10 in utils/hash.js; security tests.	Met
NFR-007	CSRF check on refresh and logout; cookie attributes; security tests. Secure attribute only when NODE_ENV=production.	Met
NFR-008	Ticket suite (41 tests) covers signature validation and tamper rejection.	Met
NFR-009	Tenant suites (84 tests).	Met
NFR-010	Parameterized queries throughout; SQL-injection cases in security suites.	Met
NFR-011	Payment suite covers Sharp re-encoding and magic-byte rejection.	Met
NFR-012	No uptime monitoring in the pilot environment yet.	Operational target
NFR-013	Order, booking and ticket mutations run inside database transactions (code review).	Met (by code review)
NFR-014	No application code updates or deletes audit_logs or ticket_checkins rows; no database-level enforcement.	Met at application level
NFR-015	Responsive layout built with Tailwind; checked manually.	Manual verification
NFR-016	Status colour coding with text labels; checked manually.	Manual verification
NFR-017	Scanner decode time not measured.	Not yet measured
NFR-018	Domain modules under backend/src/modules.	Met (by code review)
NFR-019	Empty database migrated to 40 tables with db:migrate:all (includes migrations 006 and 008).	Met
NFR-020	Stateless access tokens and database sessions; requires Redis and shared storage for multiple instances.	Met with deployment conditions

15.3 Acceptance Criteria for the Pilot

The proposed criteria for accepting the platform for a pilot are: all automated backend test suites pass; NFR-001 to NFR-004 are confirmed under a pilot load test; and the limitations in Section 14.3 have owners and follow-up actions. The recorded backend tests passed, but NFR-002 has not been measured and pilot load-test confirmation is not recorded. Formal acceptance by the university has not been recorded, so the system should not be described as formally accepted for the pilot.

16. Conclusion

This SRS describes the Gibiyachin platform as it is implemented in the project repository. Its main design choices are manual, proof-based payments; two-party confirmation of physical handovers; HMAC-SHA256 signed QR tickets with check-in and reversal; and separation of data by university.

All 323 recorded automated backend tests passed, and Section 14.1 shows which features they cover. Other features are checked only by shell scripts or manual checks. The Chapa payment provider has not been tested against the live gateway, SMS delivery is simulated, and the performance and availability targets have not been measured. Section 14.3 lists the security and operational items that should be addressed before wider use.

The next steps are the pilot load test, the hardening items in Section 14.3, and a decision on the commission rate for in-app P2P sales.